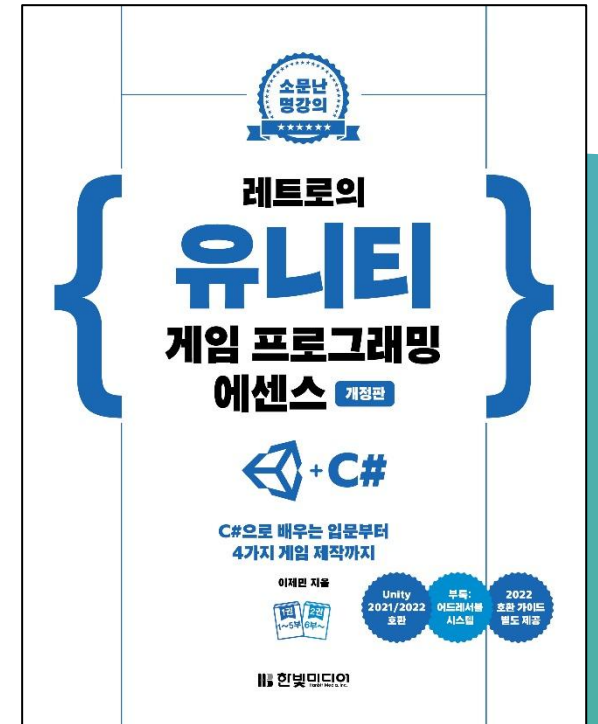


▶ Chapter 12: 유니런 배경 스크롤링과 게임 매니저

레트로의 유니티 게임프로그래밍 에센스 (개정판)



이제만 지음

보충 설명

- Rigidbody2D에서의 velocity
 - 2023이전에는 Rigidbody2D에는 velocity만 존재 (Unity6부터는 linearVelocity 추가)
 - Rigidbody3D에는 velocity, linearVelocity 모두 존재
 - 명확성을 위해 기존의 velocity를 대체하기 위해 linearVelocity 도입

Contents

- Chapter 12 유니런 배경 스크롤링과 게임 매니저
 - 12.1 배경 추가하기
 - 12.2 움직이는 배경과 발판
 - 12.3 게임 UI 만들기
 - 12.4 게임 매니저 만들기
 - 12.5 마치며



CHAPTER 12 유니런 배경 스크롤링과 게임 매니저

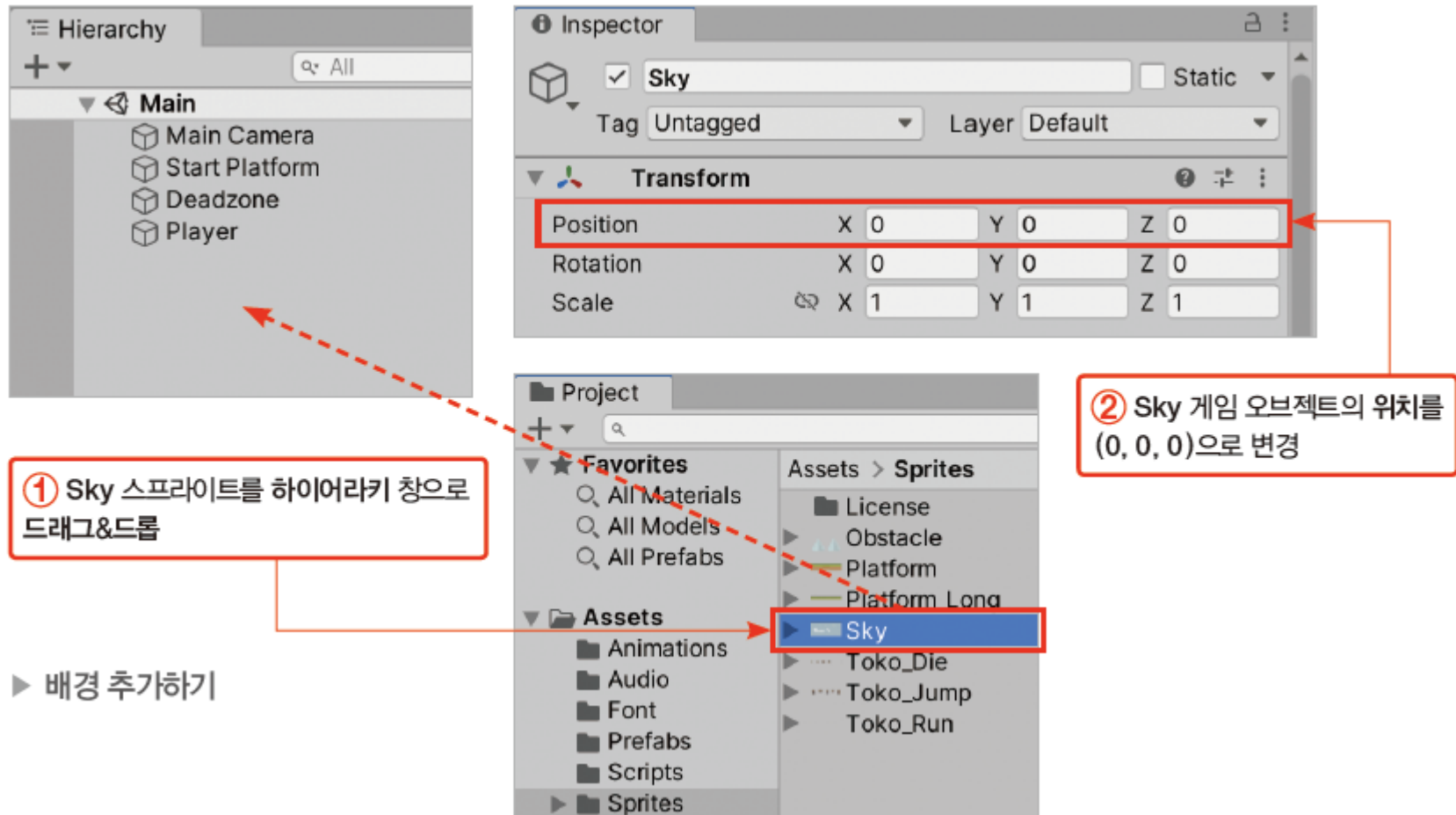
- 2D 스프라이트가 그려지는 순서 조정하기
- 반복되는 배경을 만드는 방법
- static의 개념
- 여러 해상도에서 일관된 UI를 유지하는 방법
- 싱글턴 패턴

12.1 배경 추가하기(1)

- 게임에 배경 하늘을 추가

[과정 01] 배경 추가

- ① Sprites 폴더의 Sky 스프라이트를 하이어라키 창으로 드래그&드롭
- ② Sky 게임 오브젝트의 위치를 (0, 0, 0)으로 변경

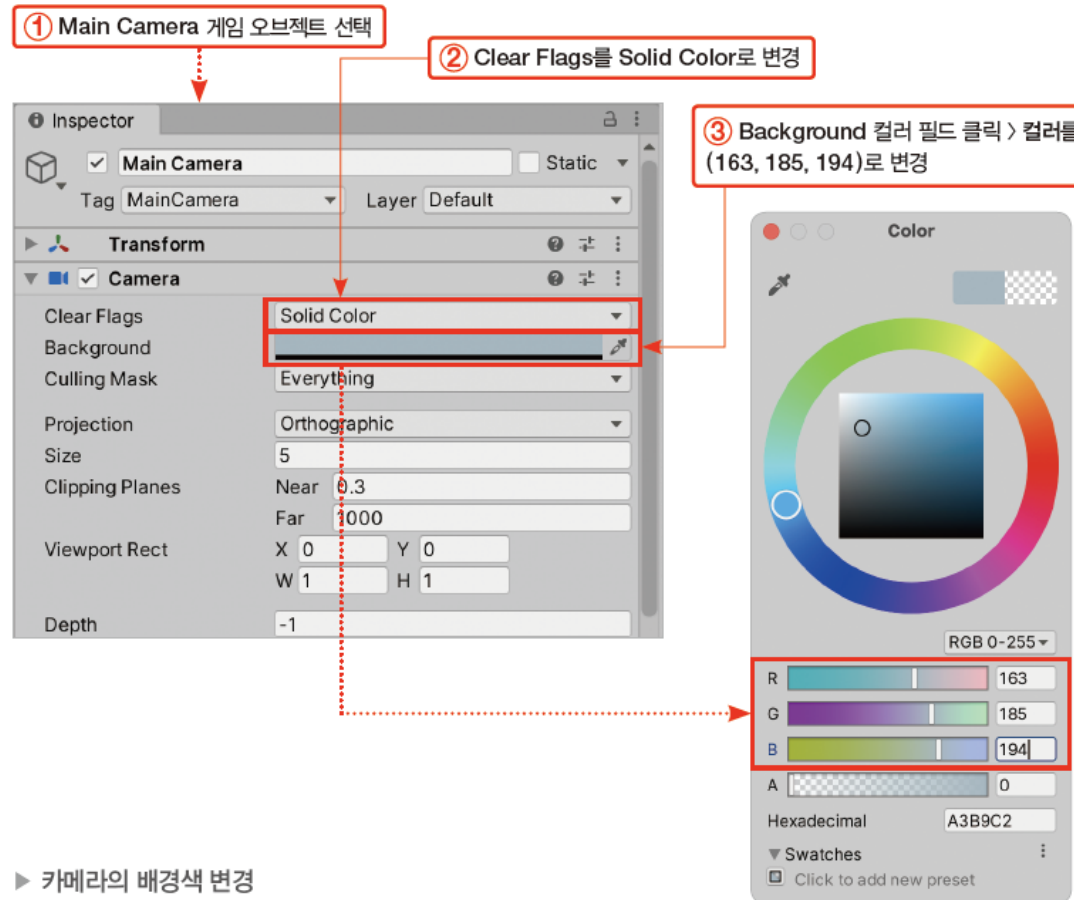


12.1 배경 추가하기(2)

- 카메라의 배경색을 Sky 스프라이트에 어울리도록 변경

[과정 02] 카메라의 배경색 변경

- ① 하이어라키 창에서 Main Camera 게임 오브젝트 선택
- ② Camera 컴포넌트의 Clear Flags를 Solid Color로 변경
- ③ Background 컬러 필드 클릭 > 컬러를 (163, 185, 194)로 변경



12.1 배경 추가하기(3)

12.1.1 정렬 레이어

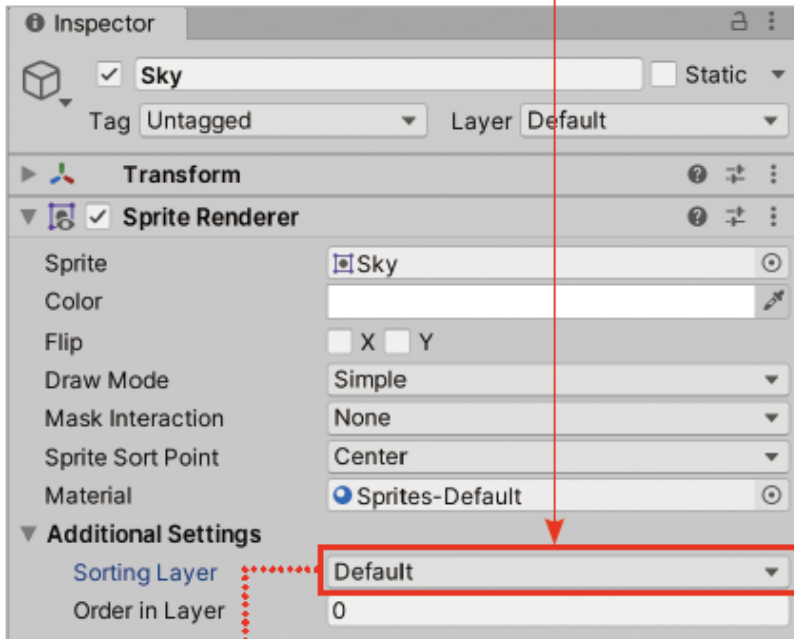
- 2D 게임 오브젝트가 그려지는 순서는 스프라이트 렌더러의 정렬 레이어가 결정
- 2D 게임 오브젝트의 스프라이트를 정렬하는 데 사용할 정렬 레이어를 추가

[과정 01] 정렬 레이어 추가하기

- ① Sprite Renderer 컴포넌트의 Sorting Layer의 Default 클릭 > Add Sorting Layer... 클릭
- ② Tags & Layers 창에서 Sorting Layers 리스트의 + 버튼 클릭
- ③ 생성된 Layer 이름을 Background로 변경
- ④ + 버튼 클릭 > 생성된 Layer 이름을 Middleground로 변경
- ⑤ + 버튼 클릭 > 생성된 Layer 이름을 Foreground로 변경

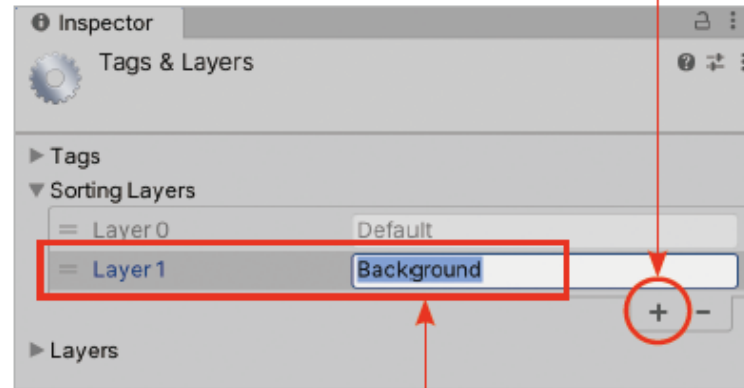
12.1 배경 추가하기(4)

① Sorting Layer의 Default 클릭 >
Add Sorting Layer... 클릭



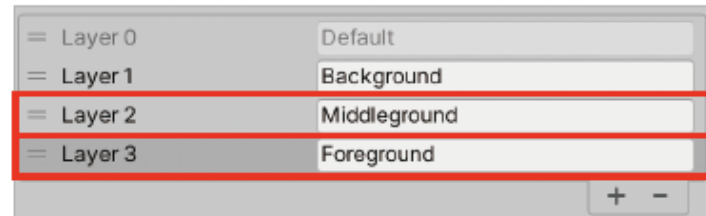
▶ 정렬 레이어 추가

② Sorting Layers 리스트의 + 버튼 클릭



③ 생성된 Layer 이름을 Background로 변경

④ + 버튼 클릭 > 생성된 Layer 이름을
Middleground로 변경



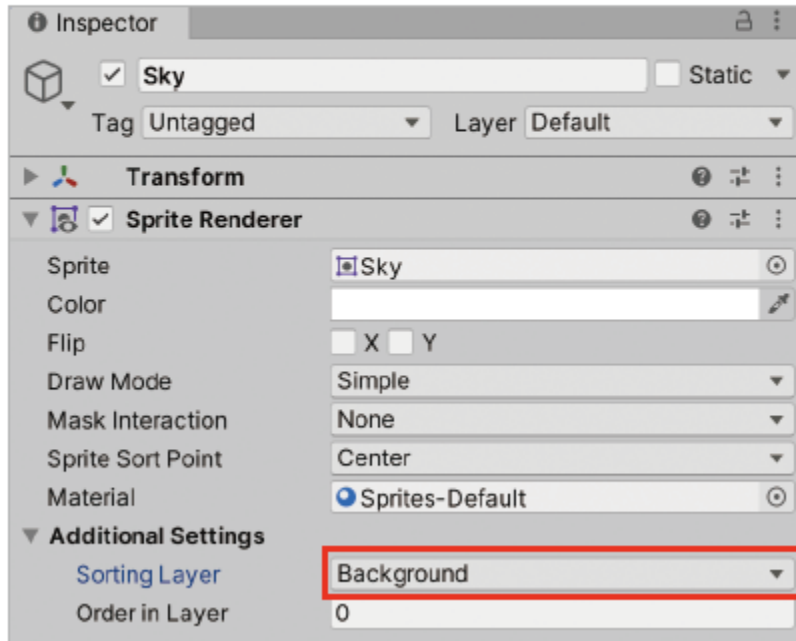
⑤ + 버튼 클릭 > 생성된 Layer 이름을
Foreground로 변경

12.1 배경 추가하기(5)

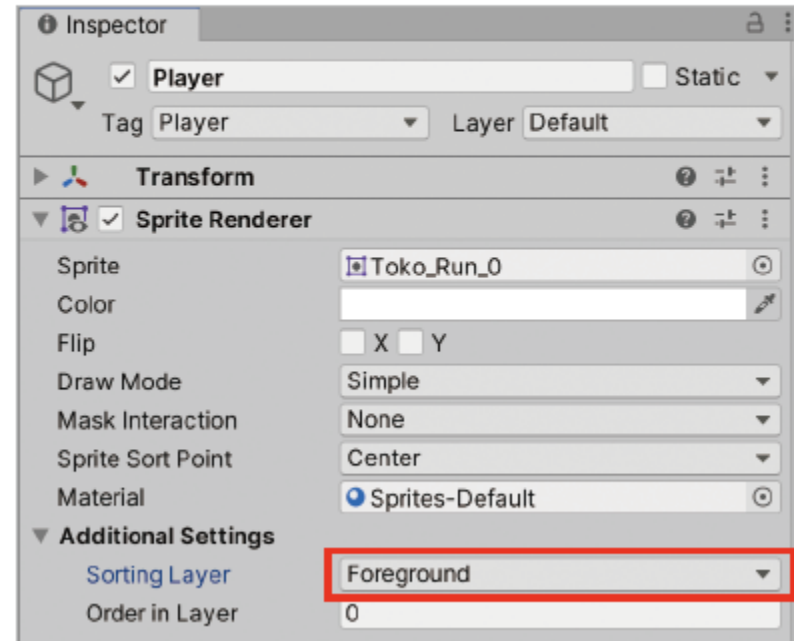
[과정 02] 정렬 레이어 할당하기

- ① 하이어라키 창에서 Sky 게임 오브젝트 선택 > Sprite Renderer 컴포넌트의 Sorting Layer를 Background로 변경
- ② Player 게임 오브젝트 선택 > Sprite Renderer 컴포넌트의 Sorting Layer를 Foreground로 변경
- ③ Start Platform 게임 오브젝트 선택 > Sprite Renderer 컴포넌트의 Sorting Layer를 Foreground로 변경

① Sky 게임 오브젝트 선택 > Sorting Layer를 Background로 변경

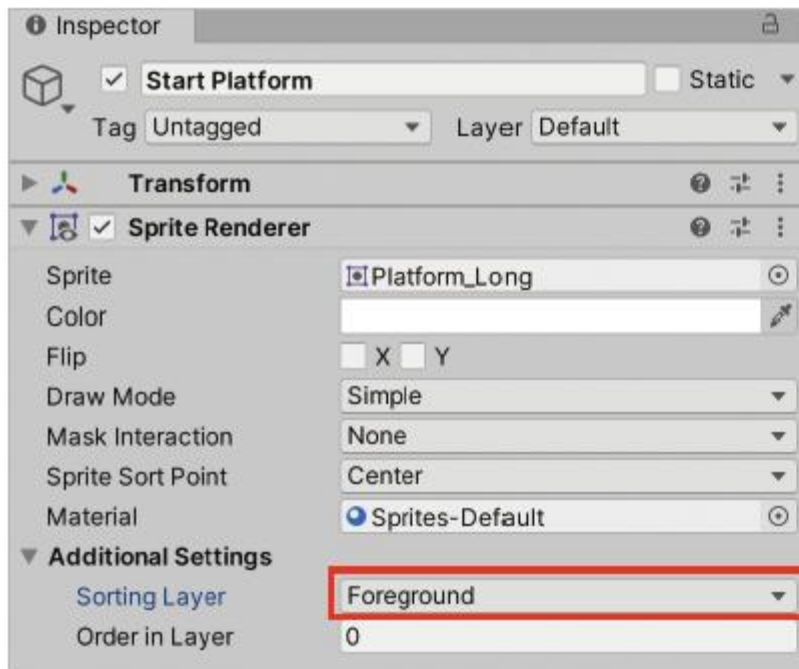


② Player 게임 오브젝트 선택 > Sorting Layer를 Foreground로 변경

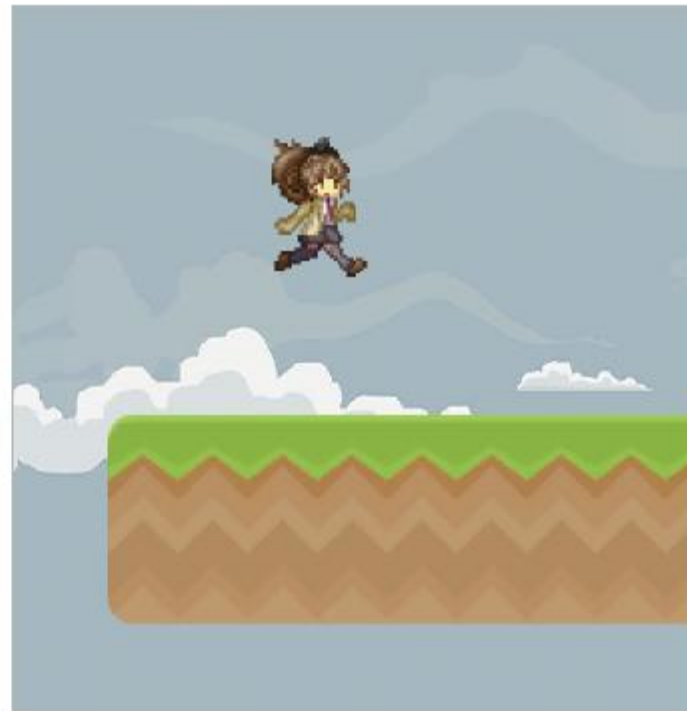


12.1 배경 추가하기(6)

③ Start Platform 게임 오브젝트 선택 >
Sorting Layer를 Foreground로 변경



▶ 정렬 레이어 할당하기



Sky가 Player와 Start Platform 뒤에 그려짐

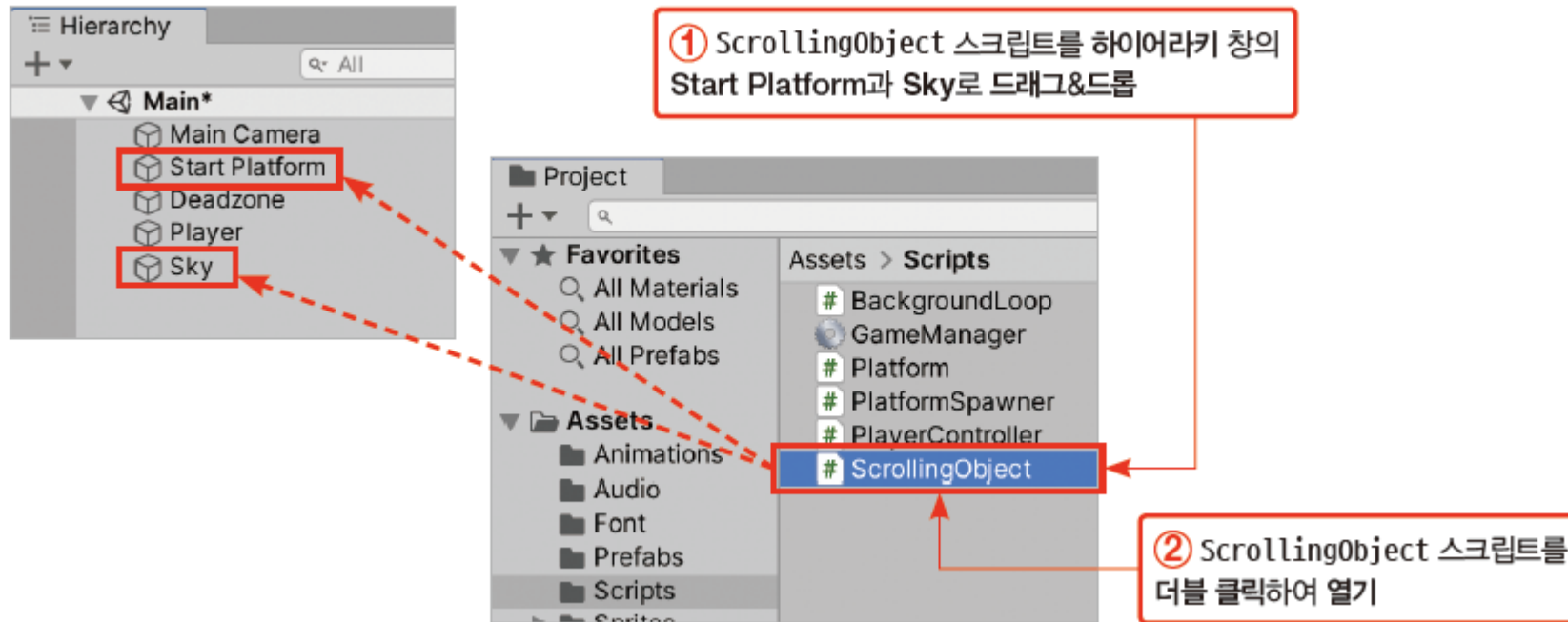
12.2 움직이는 배경과 발판(1)

12.2.1 ScrollingObject

- ScrollingObject 스크립트를 배경 Sky 게임 오브젝트와 발판 Start Platform 게임 오브젝트에 추가

[과정 01] ScrollingObject 스크립트 추가하기

- ① 프로젝트 창에서 Scripts 폴더로 이동 > ScrollingObject 스크립트를 하이어라키 창의 Start Platform과 Sky로 드래그&드롭
- ② 프로젝트 창에서 ScrollingObject 스크립트를 더블 클릭하여 열기



12.2 움직이는 배경과 발판(2)

- Update() 메서드에서 초당 speed만큼 왼쪽으로 평행이동하는 처리를 구현

[과정 02] ScrollingObject 스크립트 완성하기

① ScrollingObject 스크립트를 다음과 같이 완성

```
using UnityEngine;
```

```
// 게임 오브젝트를 계속 왼쪽으로 움직이는 스크립트
```

```
public class ScrollingObject : MonoBehaviour {
```

```
    public float speed = 10f; // 이동 속도
```

```
    private void Update() {
```

```
        // 초당 speed의 속도로 왼쪽으로 평행이동
```

```
        transform.Translate(Vector3.left * speed * Time.deltaTime);
```

```
    }
```

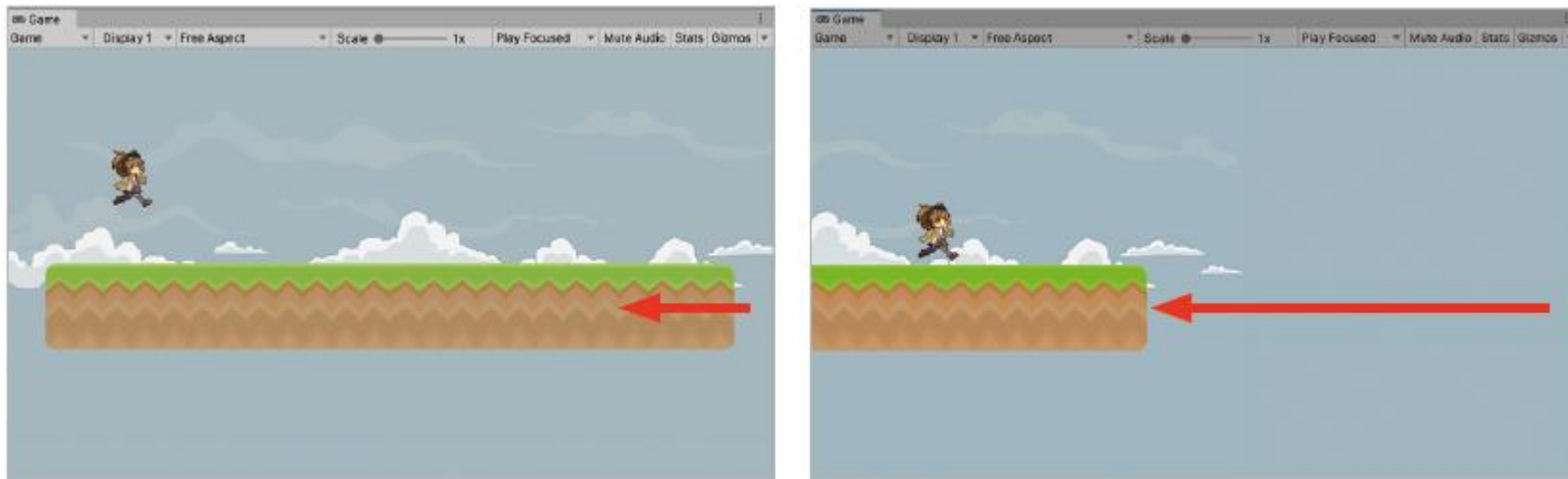
```
}
```

◀ 게임 오브젝트를 초당
(-speed, 0, 0)만큼 이동

12.2 움직이는 배경과 발판(3)

- 게임 오브젝트를 초당 (-speed, 0, 0)만큼 이동시키는 코드
 - 트랜스폼 컴포넌트의 Translate() 메서드는 평행이동 메서드
 - Translate() 메서드는 이동할 거리를 Vector3 타입으로 받음

```
transform.Translate(Vector3.left * speed * Time.deltaTime);
```

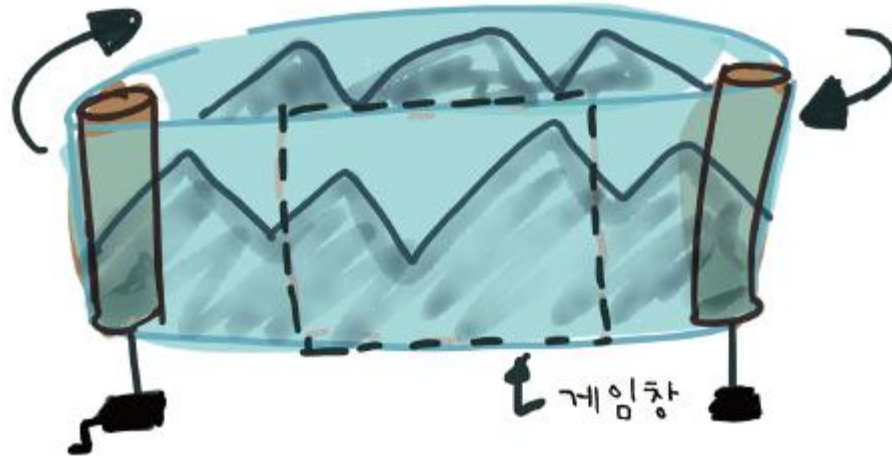


▶ ScrollingObject 테스트하기

12.2 움직이는 배경과 발판(4)

12.2.2 반복되는 배경 만들기

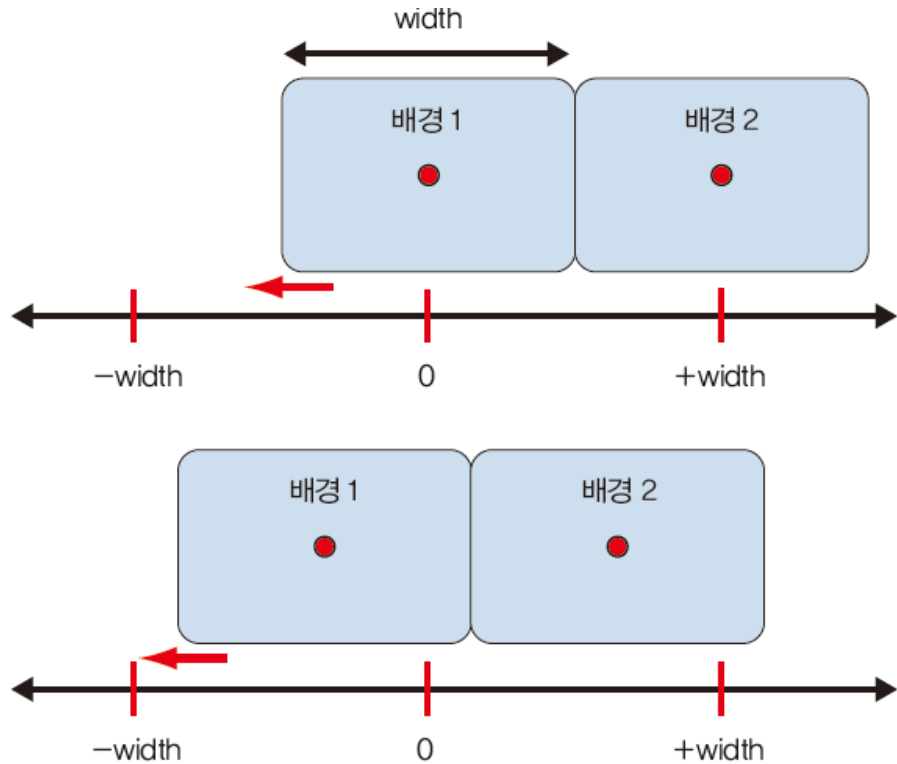
- 배경은 끊임없이 반복 스크롤링되어야 함
- 2D 게임에서 반복되는 배경을 만드는 방법은 '크랭키 박스'와 비슷



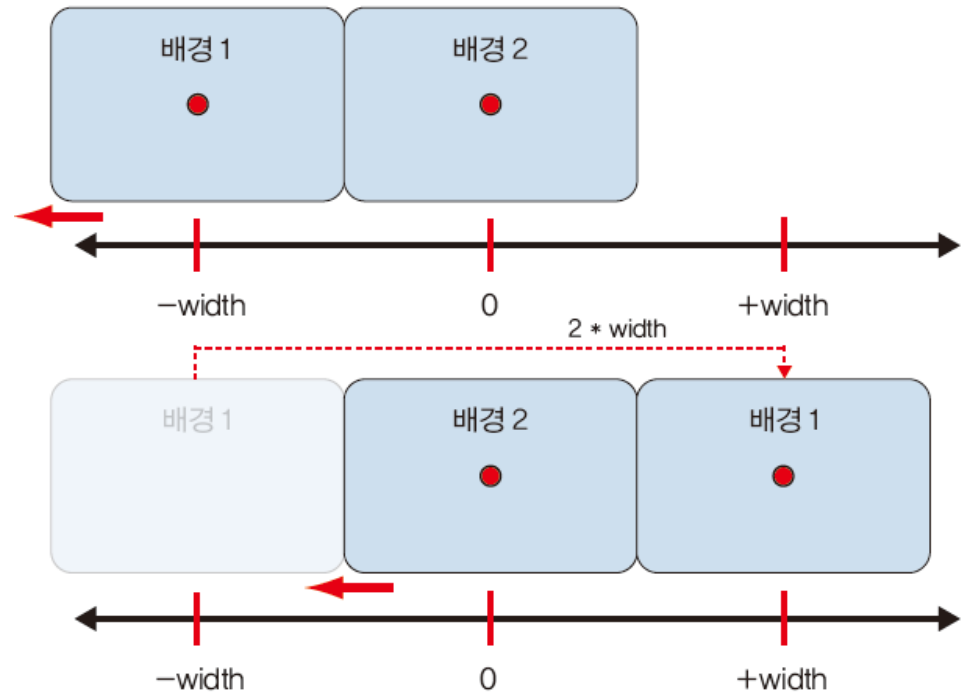
▶ 배경을 무한 스크롤링하기

12.2 움직이는 배경과 발판(5)

- 학습에서는 배경인 sky 게임 오브젝트를 두 개 이어붙여 길게 생성
 - 각 배경 게임 오브젝트는 왼쪽으로 계속 움직이다가 일정 거리 이상 움직이면 오른쪽 화면 끝으로 순간 이동



▶ 배경들의 처음 위치



▶ 재배치되는 배경

12.2 움직이는 배경과 발판(6)

- Sky 게임 오브젝트가 다른 게임 오브젝트를 물리적으로 밀어내면 안 되니 추가한 박스 콜라이더 2D 컴포넌트는 트리거로 설정

[과정 01] 박스 콜라이더 2D와 BackgroundLoop 준비하기

- ① 하이어라키 창에서 Sky 게임 오브젝트 선택
- ② Sky 게임 오브젝트에 Box Collider 2D 컴포넌트 추가(Add Component > Physics 2D > Box Collider 2D)
- ③ Box Collider 2D 컴포넌트의 Is Trigger 체크
- ④ Scripts 폴더에서 BackgroundLoop 스크립트를 하이어라키 창의 Sky로 드래그&드롭
- ⑤ BackgroundLoop 스크립트를 더블 클릭으로 열기

12.2 움직이는 배경과 발판(7)

① Sky 게임 오브젝트 선택

② Box Collider 2D 컴포넌트 추가
(Add Component > Physics 2D > Box Collider 2D)

③ Is Trigger 체크

④ BackgroundLoop 스크립트를 하이어라키 창
의 Sky로 드래그&드롭

⑤ BackgroundLoop 스크립트를
더블 클릭으로 열기

▶ 박스 콜라이더 2D와 BackgroundLoop 준비하기

The image shows the Unity interface with the following elements visible:

- Hierarchy:** A list of objects including Main Camera, Start Platform, Deadzone, Player, and Sky. The 'Sky' object is highlighted with a red box.
- Inspector:** The 'Sky' object is selected. The 'Box Collider 2D' component is expanded, and the 'Is Trigger' checkbox is checked. The 'BackgroundLoop' script is also visible in the Inspector.
- Project:** The 'BackgroundLoop' script is highlighted in the 'Scripts' folder.

12.2 움직이는 배경과 발판(8)

- width는 배경 Sky 게임 오브젝트의 가로 길이가 할당될 변수이며 Awake() 메서드에서 계산
- Awake() 메서드는 Start() 메서드처럼 초기 1회 자동 실행되는 유니티 이벤트 메서드지만, Start() 메서드보다 실행 시점이 한 프레임 더 빠름
- Update() 메서드에서는 매 프레임 현재 위치를 검사 만약 게임 오브젝트가 -width보다 더 왼쪽으로 이동하면 게임 오브젝트를 재배치하는 메서드를 실행
- Reposition()은 재배치를 실제로 실행하는 메서드이며 Update() 메서드에 의해 실행

```
using UnityEngine;
```

```
// 왼쪽 끝으로 이동한 배경을 오른쪽 끝으로 재배치하는 스크립트
```

```
public class BackgroundLoop : MonoBehaviour {
```

```
    private float width; // 배경의 가로 길이
```

```
    private void Awake() {
```

```
        // 가로 길이를 측정하는 처리
```

```
    }
```

```
    private void Update() {
```

```
        // 현재 위치가 원점에서 왼쪽으로 width 이상 이동했을 때 위치를 재배치
```

```
    }
```

```
    // 위치를 재배치하는 메서드
```

```
    private void Reposition() {
```

```
    }
```

```
}
```

12.2 움직이는 배경과 발판(9)

Awake() 메서드

[과정 01] BackgroundLoop 스크립트의 Awake () 메서드 완성하기

① Awake() 메서드 부분을 다음과 같이 완성

```
private void Awake() {  
    // BoxCollider2D 컴포넌트의 Size 필드의 x 값을 가로 길이로 사용  
    BoxCollider2D backgroundCollider = GetComponent<BoxCollider2D>();  
    width = backgroundCollider.size.x;  
}
```

12.2 움직이는 배경과 발판(10)

Update() 메서드

[과정 01] BackgroundLoop 스크립트의 Update () 메서드 완성하기

① Update() 메서드 부분을 다음과 같이 완성

```
private void Update() {  
    // 현재 위치가 원점에서 왼쪽으로 width 이상 이동했을 때 위치를 재배치  
    if (transform.position.x <= -width)  
    {  
        Reposition();  
    }  
}
```

12.2 움직이는 배경과 발판(11)

Reposition() 메서드

[과정 01] BackgroundLoop 스크립트의 Reposition () 메서드 완성하기

① Reposition() 메서드 부분을 다음과 같이 완성

```
private void Reposition() {  
    // 현재 위치에서 오른쪽으로 가로 길이 * 2만큼 이동  
    Vector2 offset = new Vector2(width * 2f, 0);  
    transform.position = (Vector2) transform.position + offset;  
}
```

12.2 움직이는 배경과 발판(12)

전체 BackgroundLoop 스크립트

```
using UnityEngine;

// 왼쪽 끝으로 이동한 배경을 오른쪽 끝으로 재배치하는 스크립트
public class BackgroundLoop : MonoBehaviour {
    private float width; // 배경의 가로 길이

    private void Awake() {
        // BoxCollider2D 컴포넌트의 Size 필드의 x 값을 가로 길이로 사용
        BoxCollider2D backgroundCollider = GetComponent<BoxCollider2D>();
        width = backgroundCollider.size.x;
    }
}
```

```
private void Update() {
    // 현재 위치가 원점에서 왼쪽으로 width 이상 이동했을 때 위치를 재배치
    if (transform.position.x <= -width)
    {
        Reposition();
    }
}

// 위치를 재배치하는 메서드
private void Reposition() {
    // 현재 위치에서 오른쪽으로 가로 길이 * 2만큼 이동
    Vector2 offset = new Vector2(width * 2f, 0);
    transform.position = (Vector2) transform.position + offset;
}
}
```

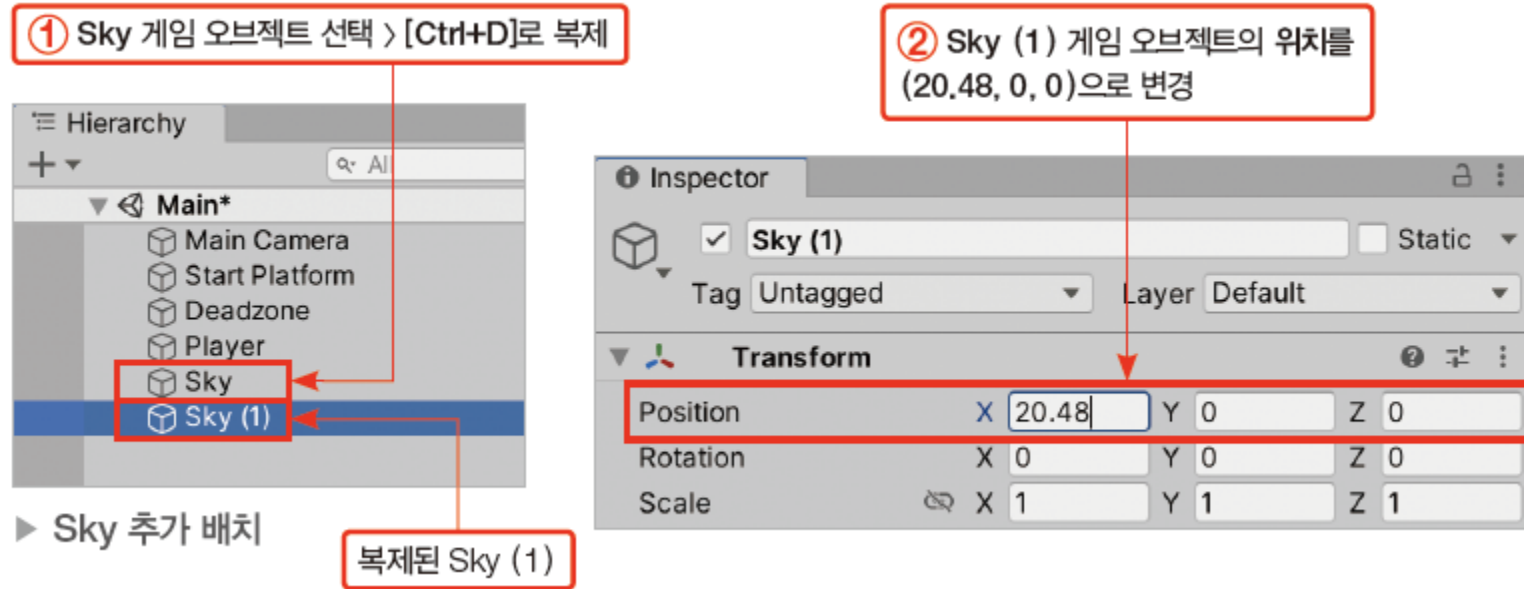
12.2 움직이는 배경과 발판(13)

배경 스크롤링 완성

- Sky 게임 오브젝트를 두 개 만들어 이어붙이고 무한 스크롤링되는 배경을 완성

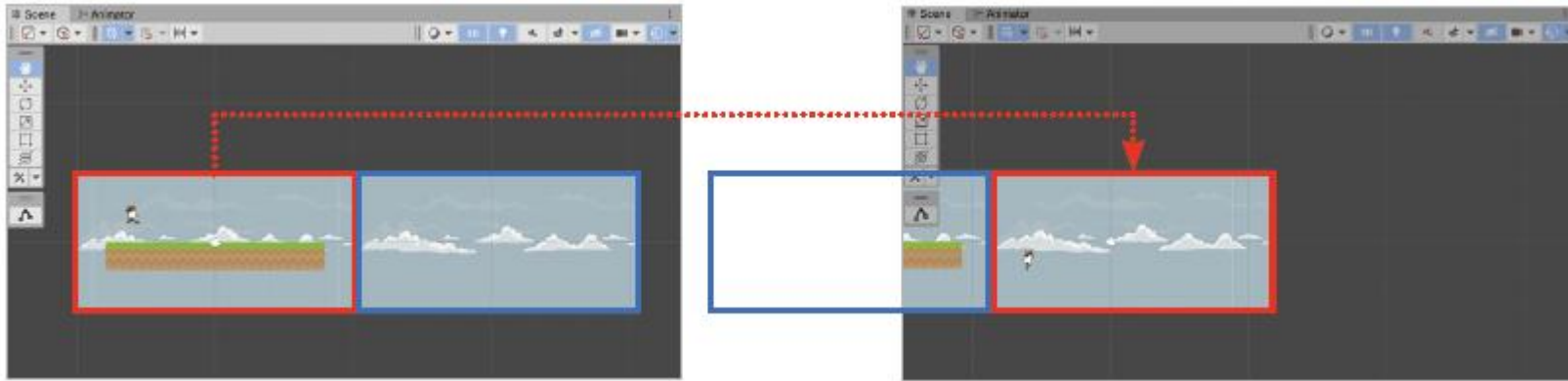
[과정 01] Sky 추가 배치

- ① 하이어라키 창에서 Sky 게임 오브젝트 선택 > [Ctrl+D]로 복제
- ② 복제된 Sky (1) 게임 오브젝트의 위치를 (20.48, 0, 0)으로 변경



12.2 움직이는 배경과 발판(14)

- 플레이 버튼을 눌러 배경이 제대로 반복되는지 테스트



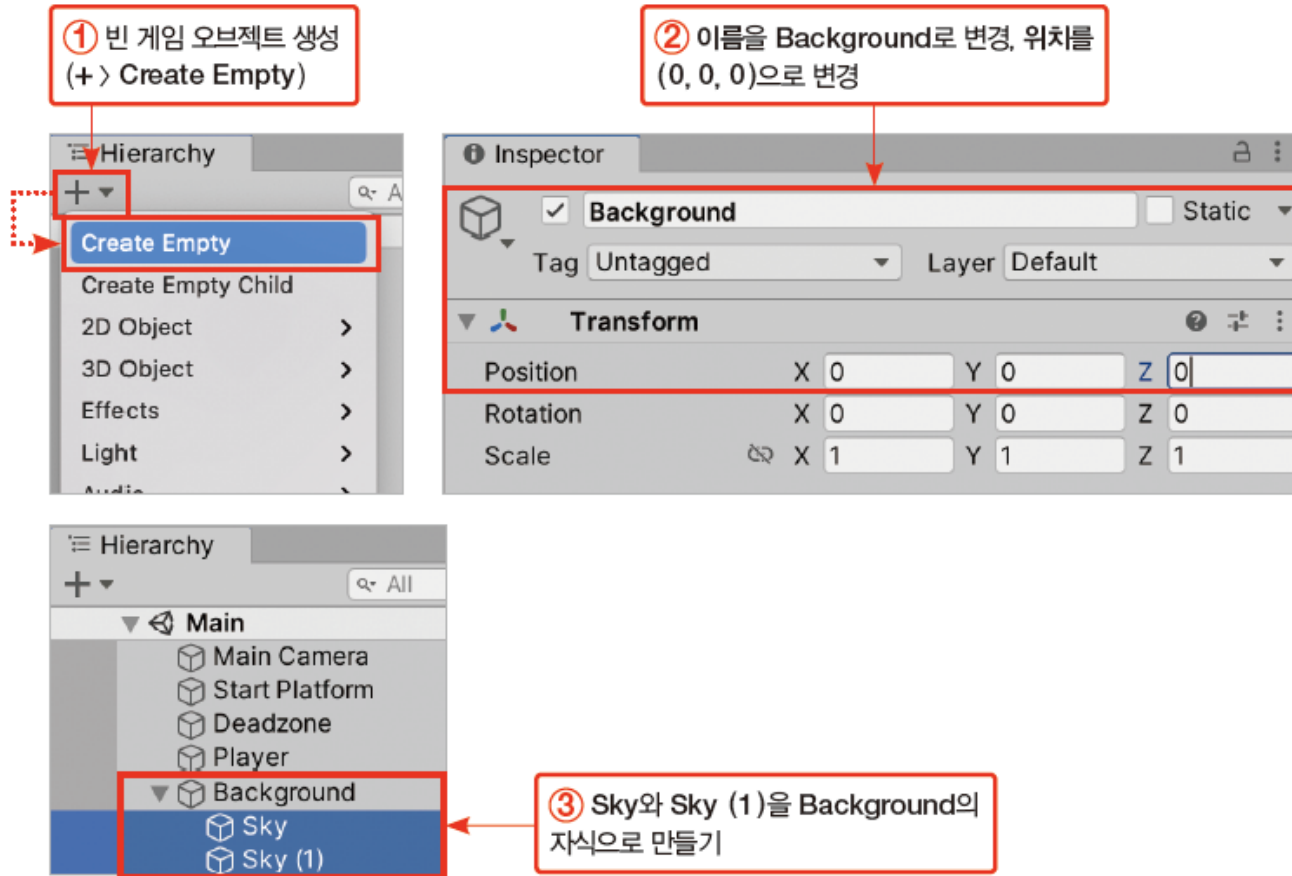
▶ 반복되는 배경

12.2 움직이는 배경과 발판(15)

- 두 개의 Sky 게임 오브젝트를 하나의 게임 오브젝트의 자식으로 만들어 하이어라키 창을 정리

[과정 02] 배경 오브젝트 정리하기

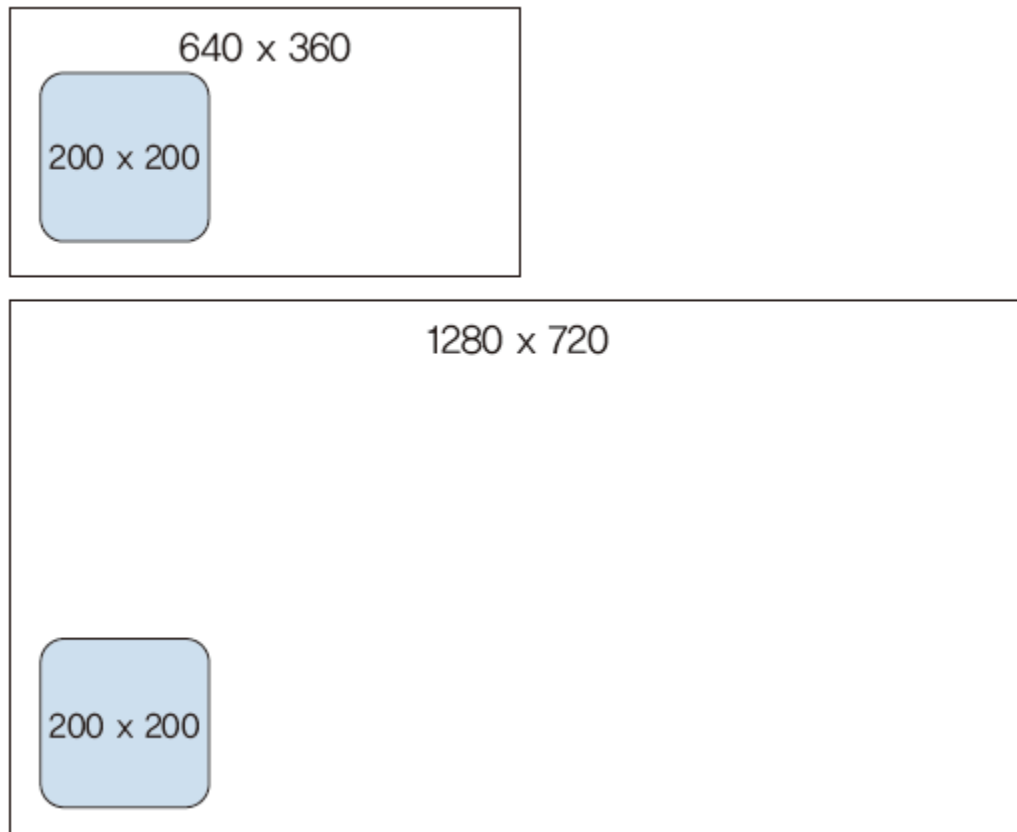
- ① 빈 게임 오브젝트 생성(+ > Create Empty)
- ② 생성된 게임 오브젝트의 이름을 Background로 변경, 위치를 (0, 0, 0)으로 변경
- ③ 하이어라키 창에서 Sky와 Sky (1)을 Background로 드래그&드롭하여 Background의 자식으로 만들기



12.3 게임 UI 만들기(1)

12.3.1 고정 픽셀 크기

- 점수와 게임오버 메시지를 표시하는 UI 만들기
- 캔버스의 크기는 게임을 실행 중인 화면의 해상도로 결정

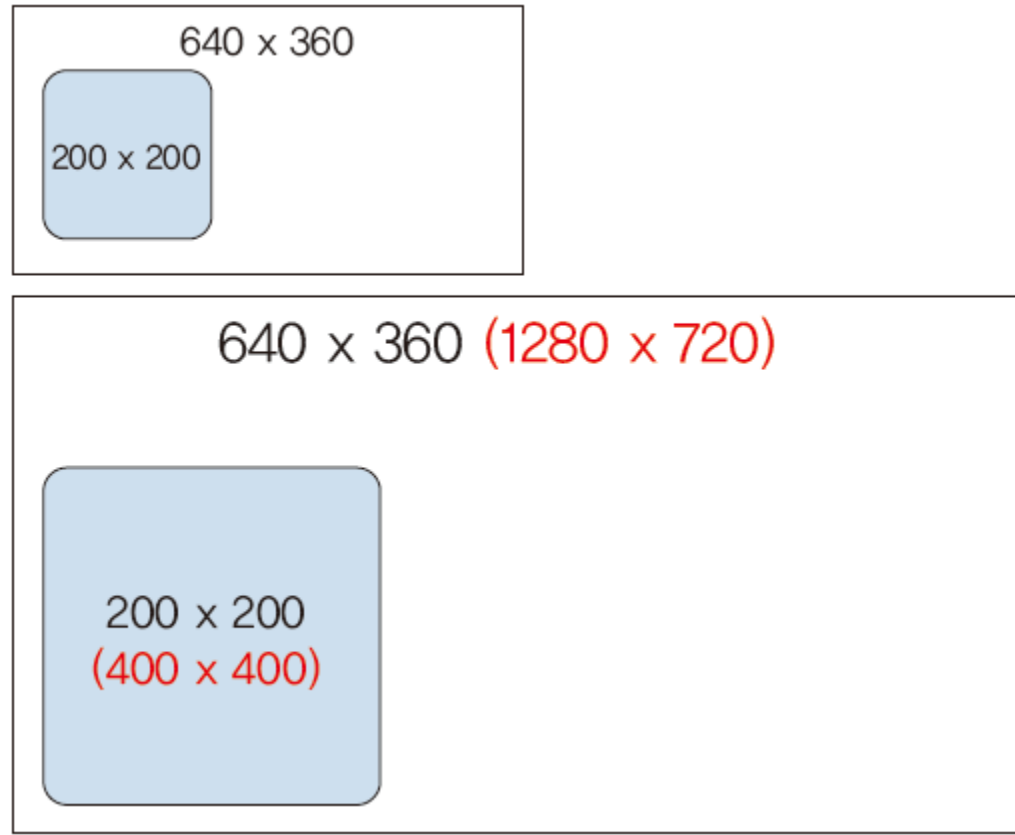


▶ 고정 픽셀 크기

12.3 게임 UI 만들기(2)

12.3.2 화면 크기에 따라 스케일

- '화면 크기에 따라 스케일(Scale With Screen Size)' 모드를 UI 스케일 모드로 사용
 - 기준 화면 크기를 정하고, 실행 화면이 기준 화면보다 크거나 작을 때는 자동으로 확대/축소(스케일업/스케일다운)

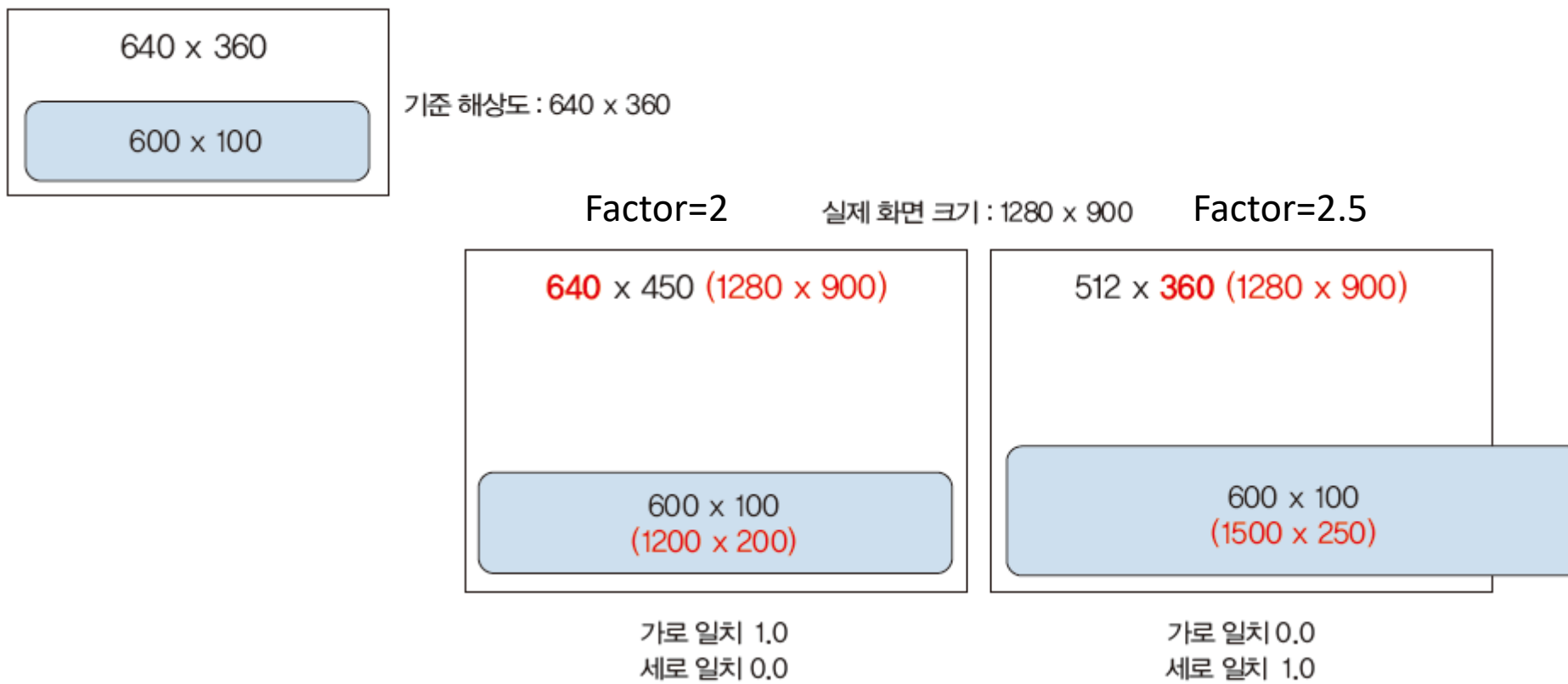


▶ 다양한 크기의 해상도에서도 동일한 UI 배치를 유지함

12.3 게임 UI 만들기(3)

방향 매치

- "화면 크기에 따라 스케일" 모드는 실제 화면과 기준 해상도 사이의 화면 비율이 다른 경우
 - 캔버스 스케일러 컴포넌트의 일치Match 필드 값이 높은 방향의 길이를 유지하고 다른 방향의 길이를 조절



12.3 게임 UI 만들기(4)

12.3.3 캔버스 스케일러 설정

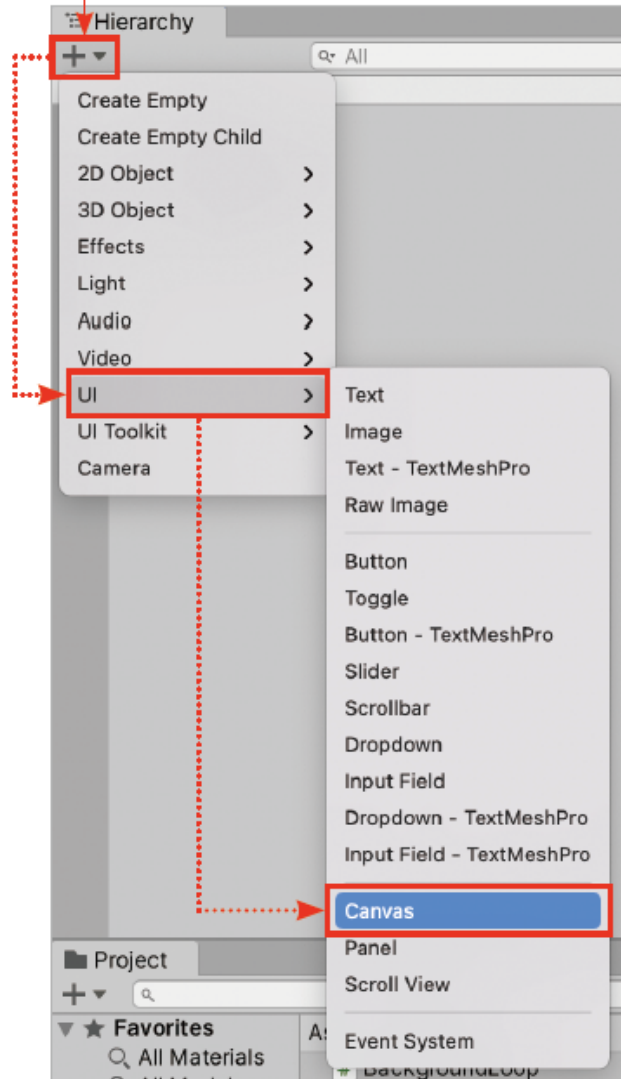
- '화면 크기에 따라 스케일' 모드를 사용해 640 × 360을 기준 해상도로 사용하여 UI를 배치
- 캔버스 스케일 모드는 캔버스 스케일러 컴포넌트에서 설정

[과정 01] 캔버스를 만들고 기준 해상도 사용

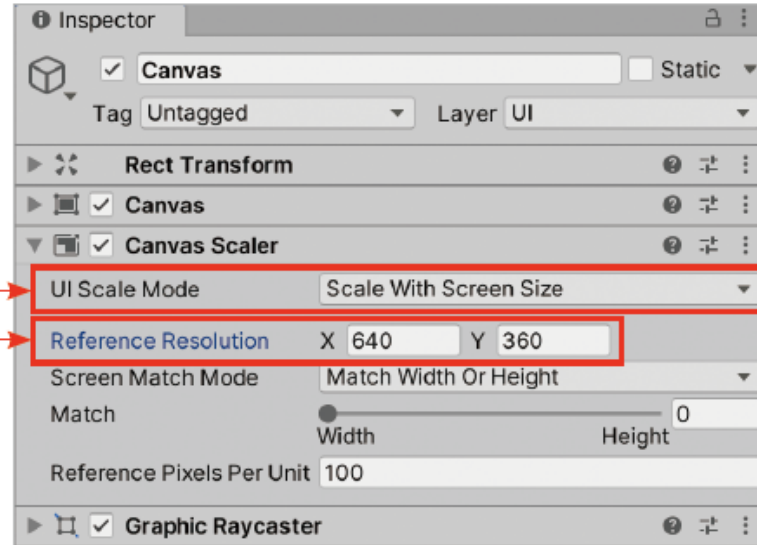
- ① Canvas 게임 오브젝트 생성(+ > UI > Canvas)
- ② Canvas Scaler 컴포넌트의 UI Scale Mode를 Scale With Screen Size로 변경
- ③ Reference Resolution을 (640, 360)으로 변경

12.3 게임 UI 만들기(5)

① Canvas 게임 오브젝트 생성(+ > UI > Canvas)



② Canvas Scaler 컴포넌트의 UI Scale Mode를 Scale With Screen Size로 변경



③ Reference Resolution을 (640, 360)으로 변경

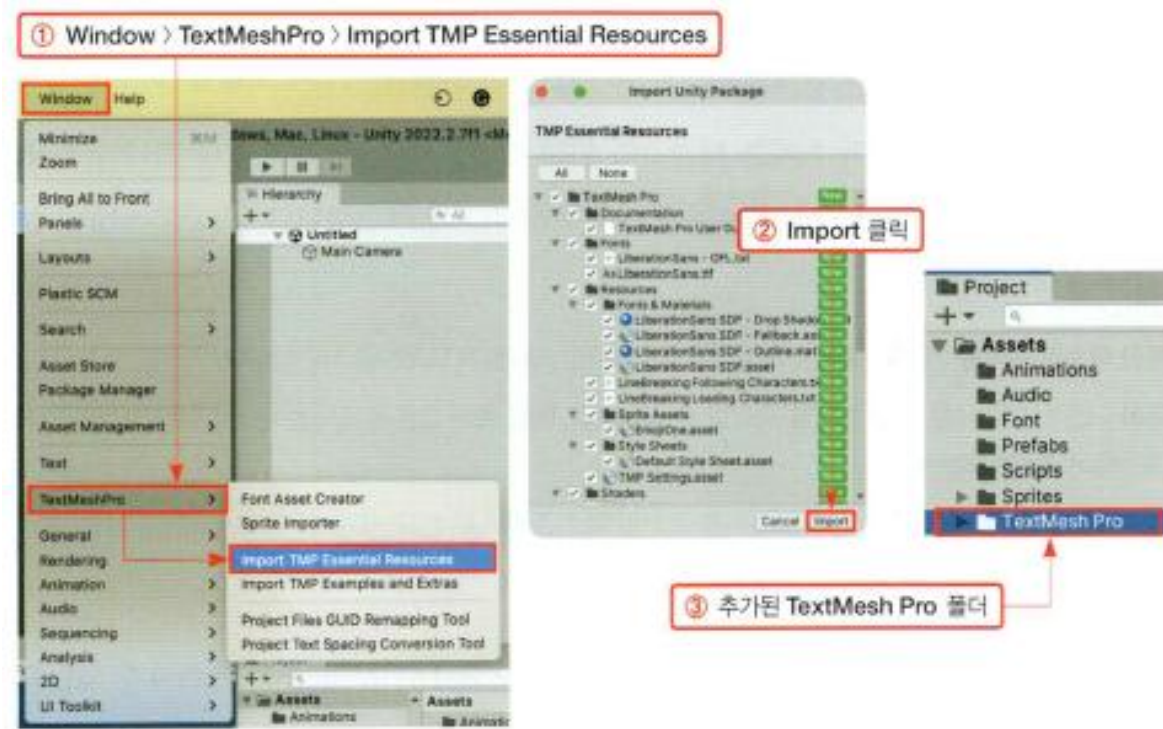
12.3 게임 UI 만들기(5)

12.3.4 텍스트 메시 프로(TextMeshPro UI)

- 텍스트 메시 프로 설치하기

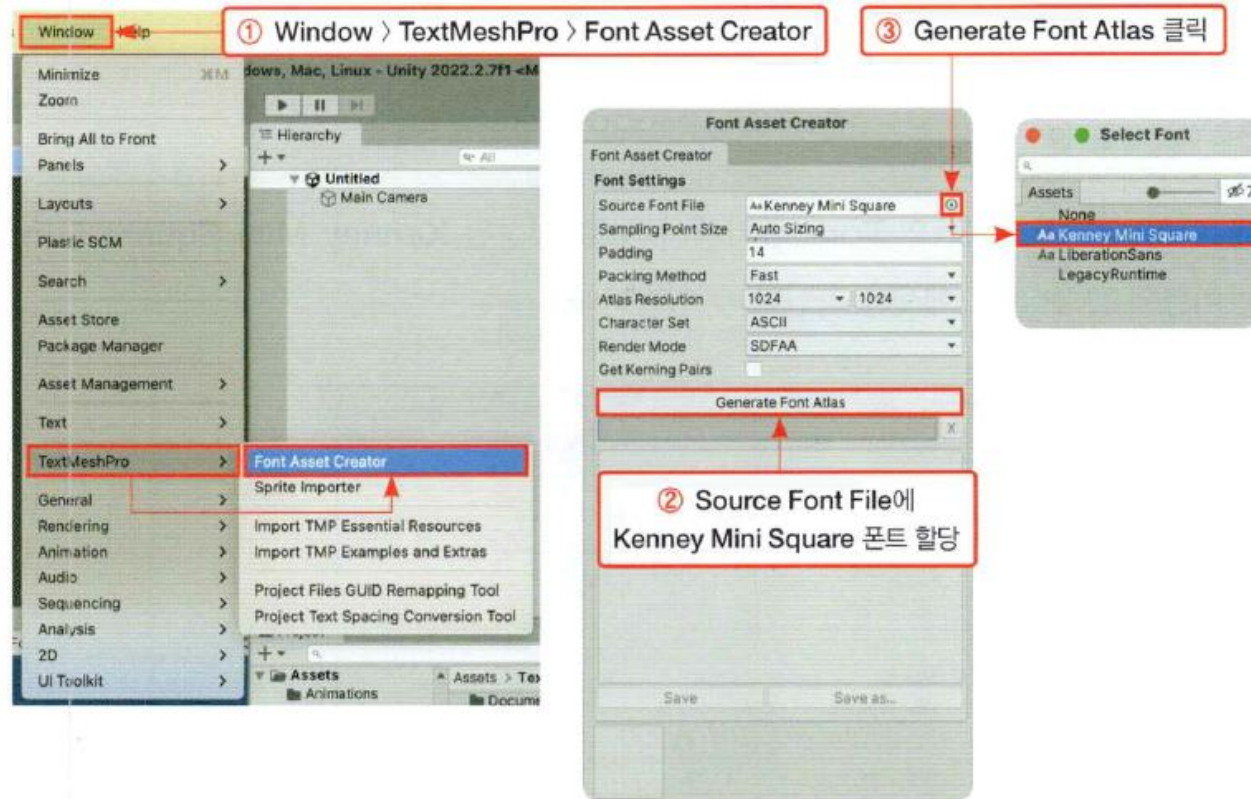
- [과정 01] 텍스트 메시 프로 필수 에셋 설치

- ① 유니티 상단 메뉴 Window>TextMeshPro>Import TMP Essential Resources 클릭
- ② 패키지 импорт 창에서 Import 클릭
- ③ Assets 폴더에 TextMeshPro 폴더가 추가된 것을 확인



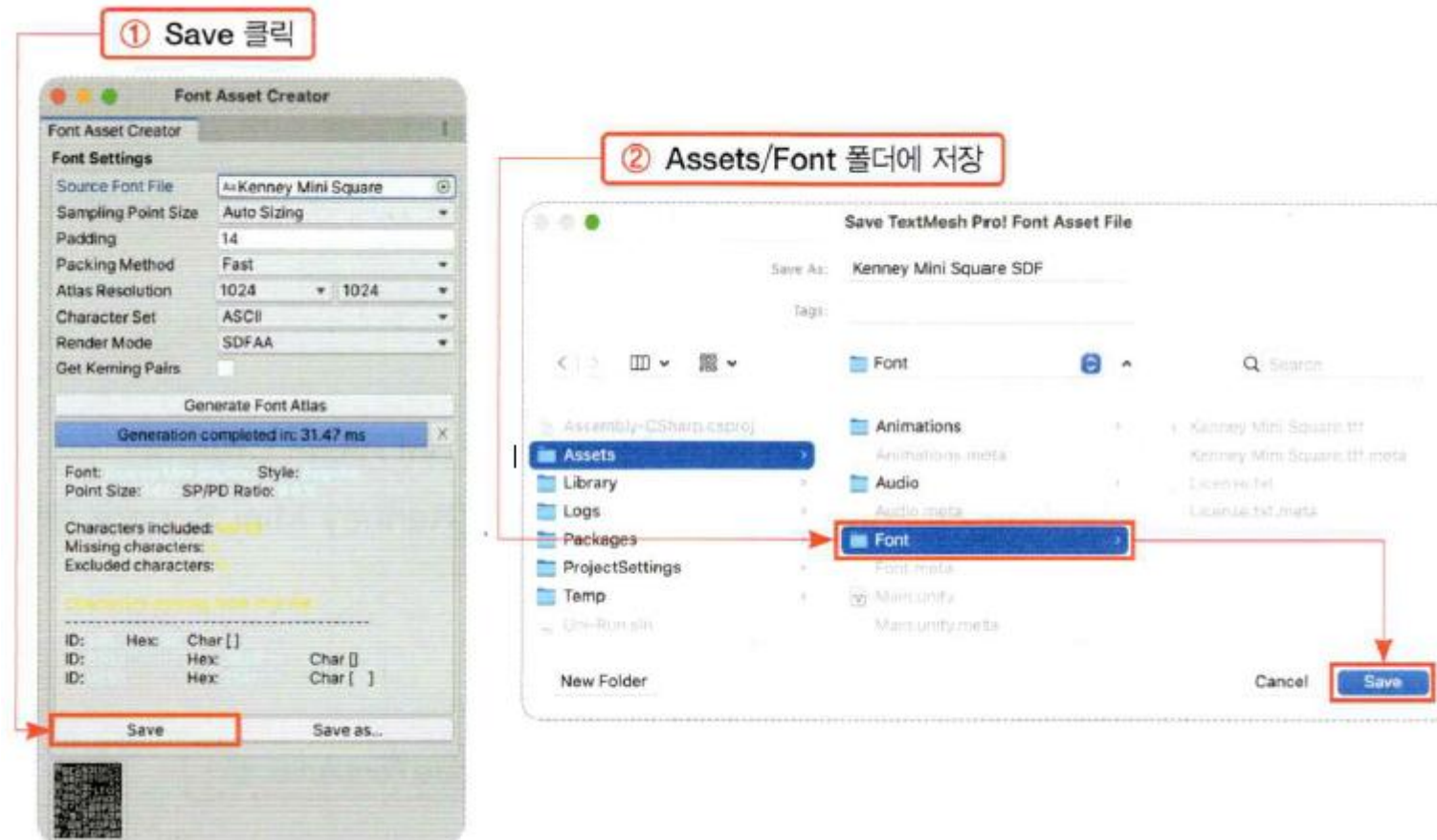
12.3 게임 UI 만들기(5)

- TMP 폰트 준비하기
- [과정 01] TMP 폰트 예셋 만들기1
 - ① 폰트 생성 창 열기(유니티 상단 메뉴의 Window>TextMeshPro>Font Asset Creator
 - ② Font Asset Creator창의 Source Font File의 선택 버튼 클릭>Kenney Mini Square 폰트 할당
 - ③ Generate Font Atlas 클릭 > 폰트 아틀라스가 생성됨

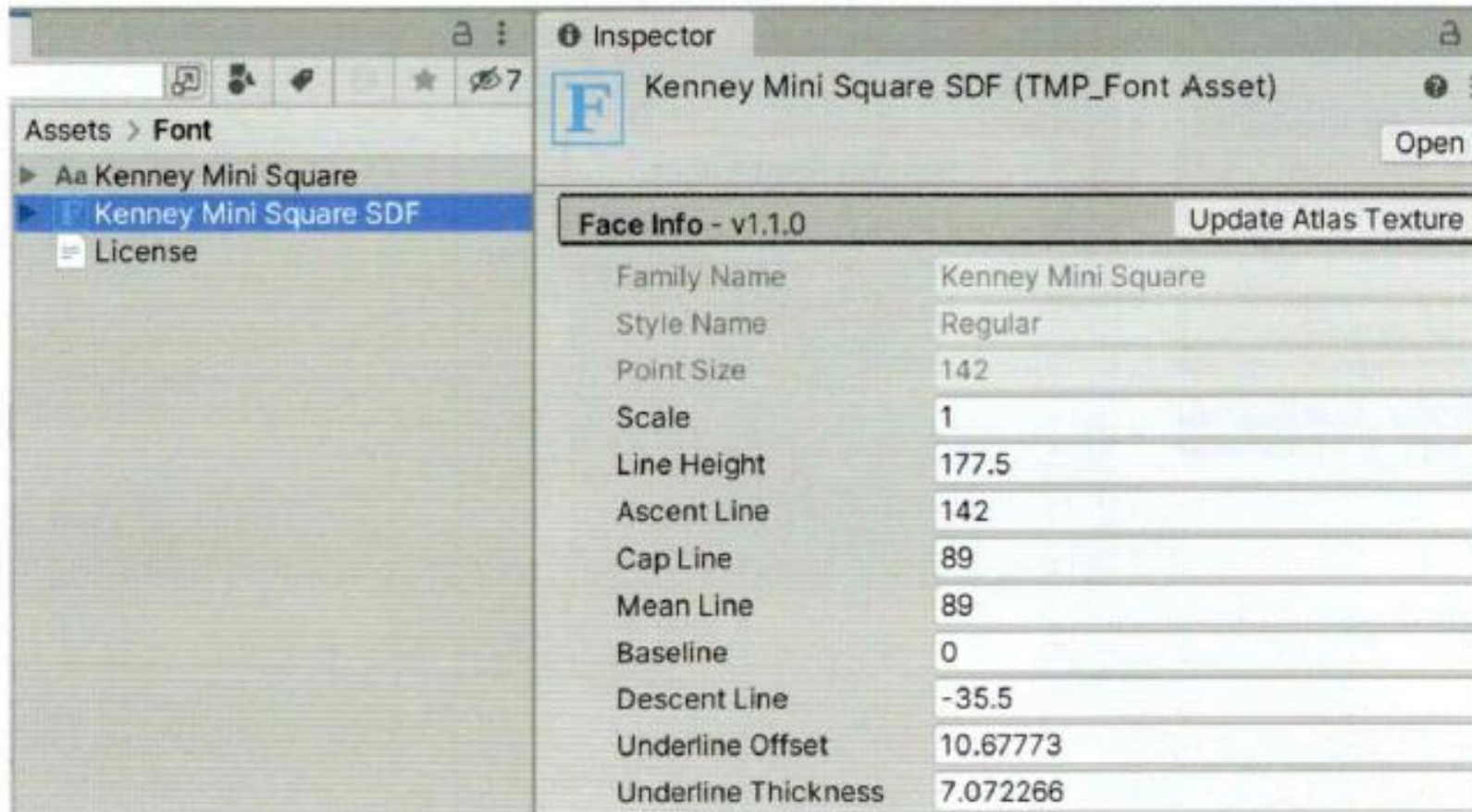


12.3 게임 UI 만들기(5)

- TMP 폰트 준비하기
- [과정 02] TMP 폰트 에셋 만들기2
 - ① Save 클릭>저장용 탐색창에서
 - ② Assets/Font 폴더에 저장



- Assets/Font 폴더에 새로 생성된 Kenney Mini Square Font SDF라는 이름의 TMP 폰트 에셋 확인



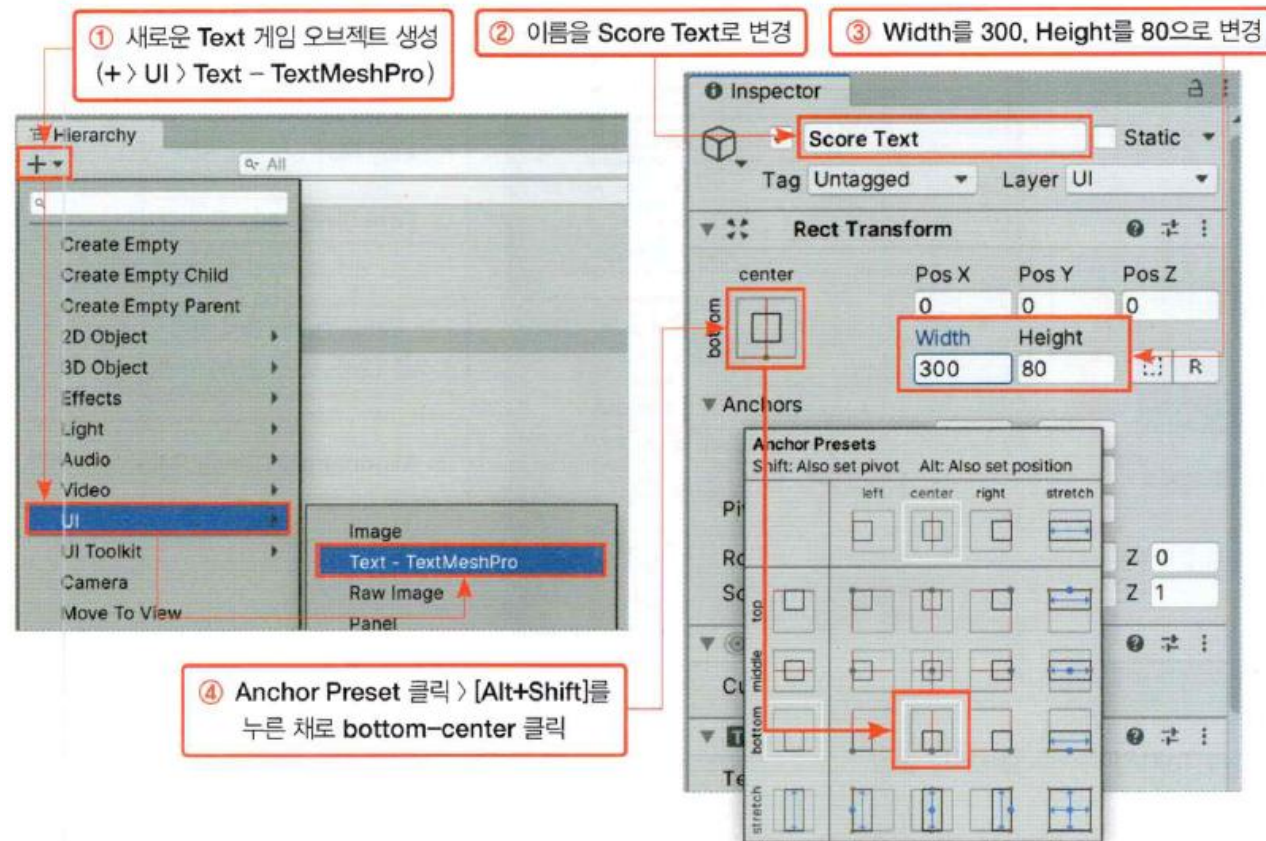
12.3 게임 UI 만들기(6)

12.3.5 점수 UI 텍스트 만들기

- 점수를 표시하기 위한 UI 텍스트 만들기

[과정 01] 점수 UI 텍스트 만들기

- ① 새로운 Text 게임 오브젝트 생성(+ > UI > Text-TextMeshPro)
- ② Text(TMP) 게임 오브젝트의 이름을 Score Text로 변경
- ③ Rect Transform 컴포넌트의 Width를 300, Height를 80으로 변경
- ④ Anchor Preset 클릭 > [Alt+Shift]를 누른 채로 bottom-center 클릭



12.3 게임 UI 만들기(7)

- Score Text 텍스트의 내용을 바꾸고 효과를 추가

[과정 02] Score Text의 텍스트 컴포넌트 설정

- ① Text 컴포넌트의 Text 필드를 Score : 0으로 변경
- ② Font 필드 옆의 선택 버튼 클릭 > 선택 창에서 Kenny Mini Square SDF 폰트 더블 클릭
- ③ Font Size를 50으로 변경
- ④ Vertex Color 필드 클릭 > 폰트 컬러를 (255, 255, 255)로 변경
- ⑤ Alignment를 Center, Middle로 변경

12.3 게임 UI 만들기(8)

The image shows the Unity Inspector and Color Picker windows. The Inspector window displays the properties of a Text object. The Color Picker window shows the RGB values for the Vertex Color.

Inspector Window:

- Score Text (Static)
- Tag: Untagged
- Layer: UI
- Rect Transform
- Canvas Renderer
- TextMeshPro - Text (UI)
- Text Input: Enable RTL Editor
- Score : 0
- Text Style: Normal
- Main Settings
 - Font Asset: Kenney Mini Square SDF (TMP_)
 - Material Preset: Kenney Mini Square SDF Material
 - Font Style: B I U S ab AB SC
 - Font Size: 50
 - Auto Size: ☐
 - Vertex Color:
 - Color Gradient: ☐
 - Override Tags: ☐
 - Spacing Options (em): Character 0 Word 0 Line 0 Paragraph 0
 - Alignment: Center, Middle
 - Wrapping: Enabled
 - Overflow: Overflow
 - Horizontal Mapping: Character
 - Vertical Mapping: Character

Color Picker Window:

- Color:
- RGB 0-255
 - R: 255
 - G: 255
 - B: 255
 - A: 255
- Hexadecimal: FFFFFFFF

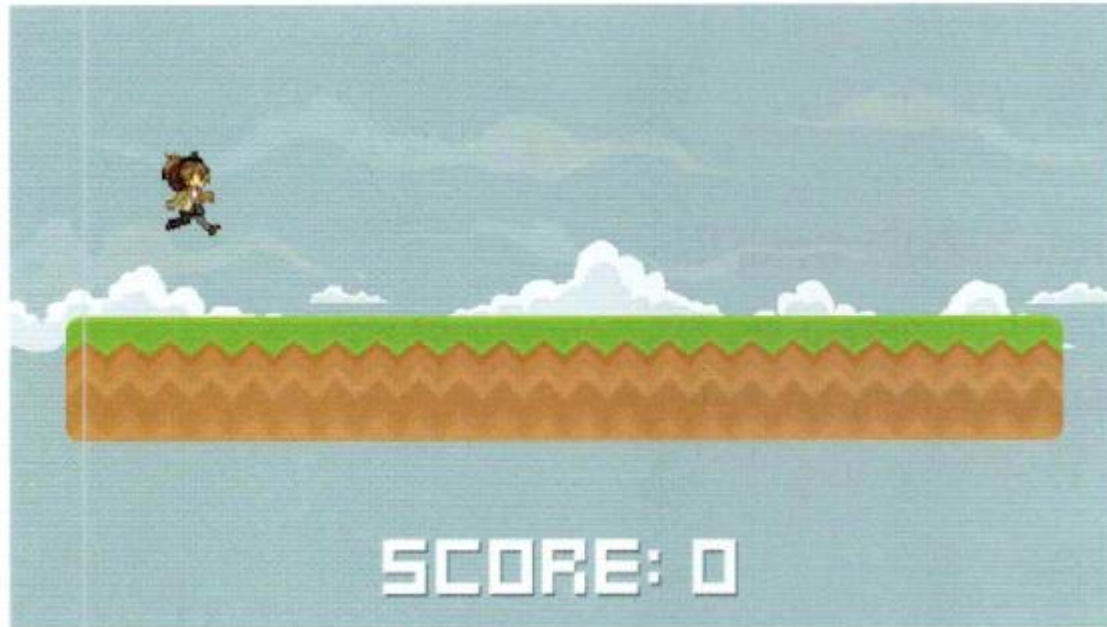
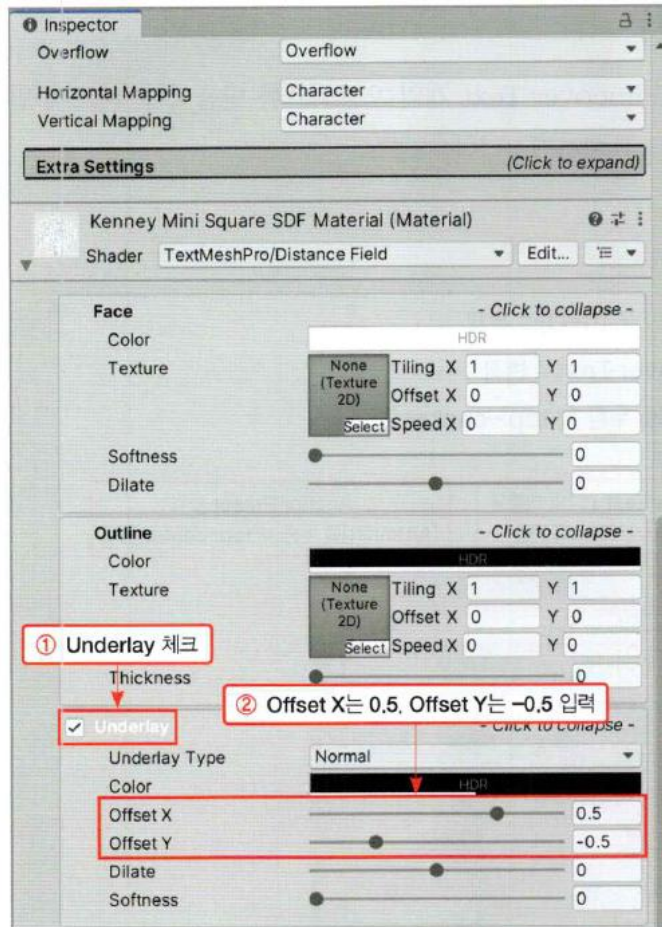
Annotations:

- ① Text 필드를 Score : 0으로 변경
- ② Font Asset 필드 옆 선택 버튼 클릭 > Kenney Mini Square SDF 폰트 더블 클릭
- ③ Font Size를 50으로 변경
- ④ Vertex Color 필드 클릭 > 컬러를 (255, 255, 255)로 변경
- ⑤ Alignment를 Center, Middle로 변경

12.3 게임 UI 만들기(9)

[과정 03] Score Text에 그림자 추가

- ① 인스펙터 창을 아래로 스크롤 > Kenney Mini Square SDF Material 설정에서 Underlay 체크
- ② Offset X는 0.5 Y는 -0.5 입력



▶ 완성된 Score Text 게임 오브젝트의 모습

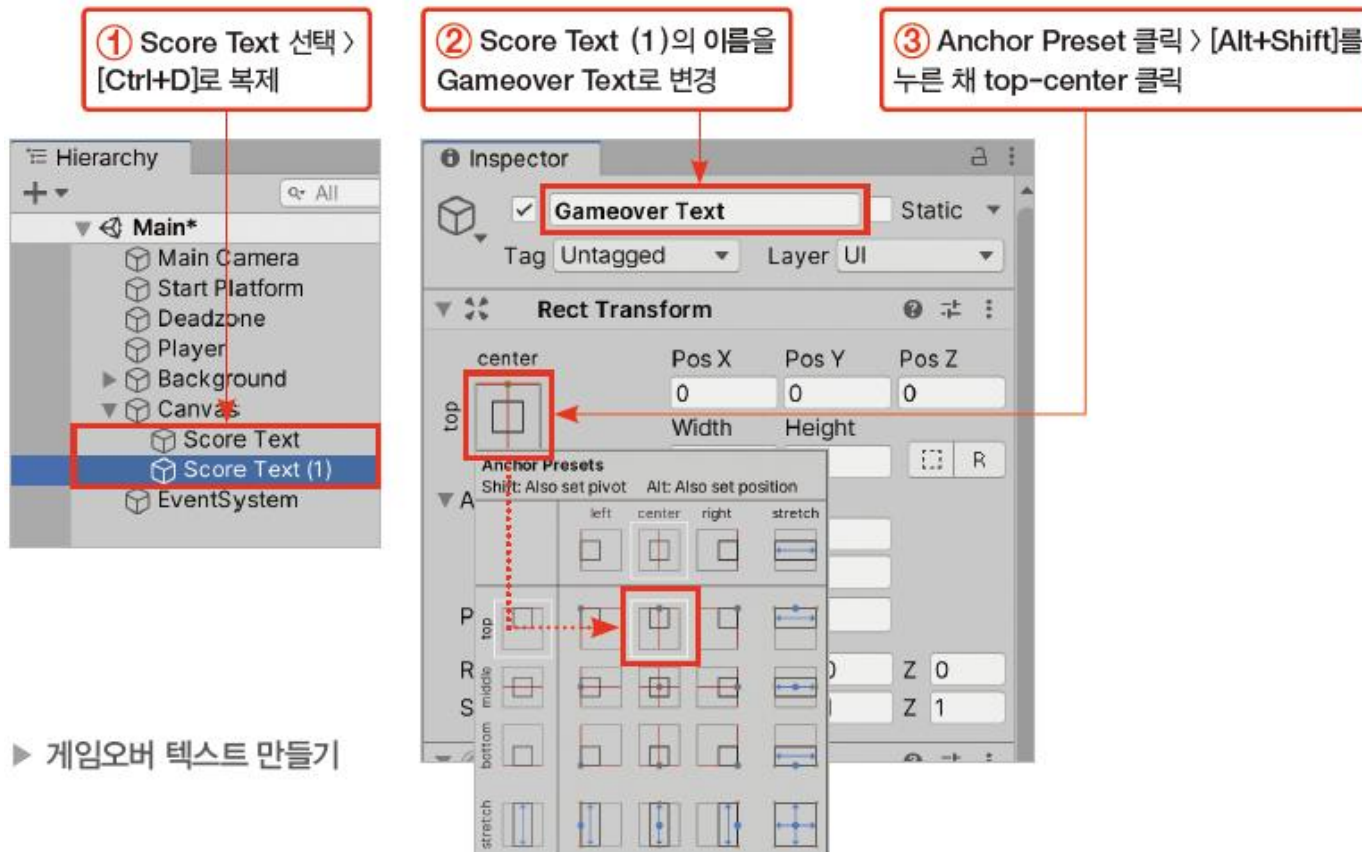
12.3 게임 UI 만들기(10)

12.3.6 게임오버 텍스트 만들기

- 게임오버 메시지를 표시할 Gameover Text 게임 오브젝트 만들기

[과정 01] 게임오버 텍스트 만들기

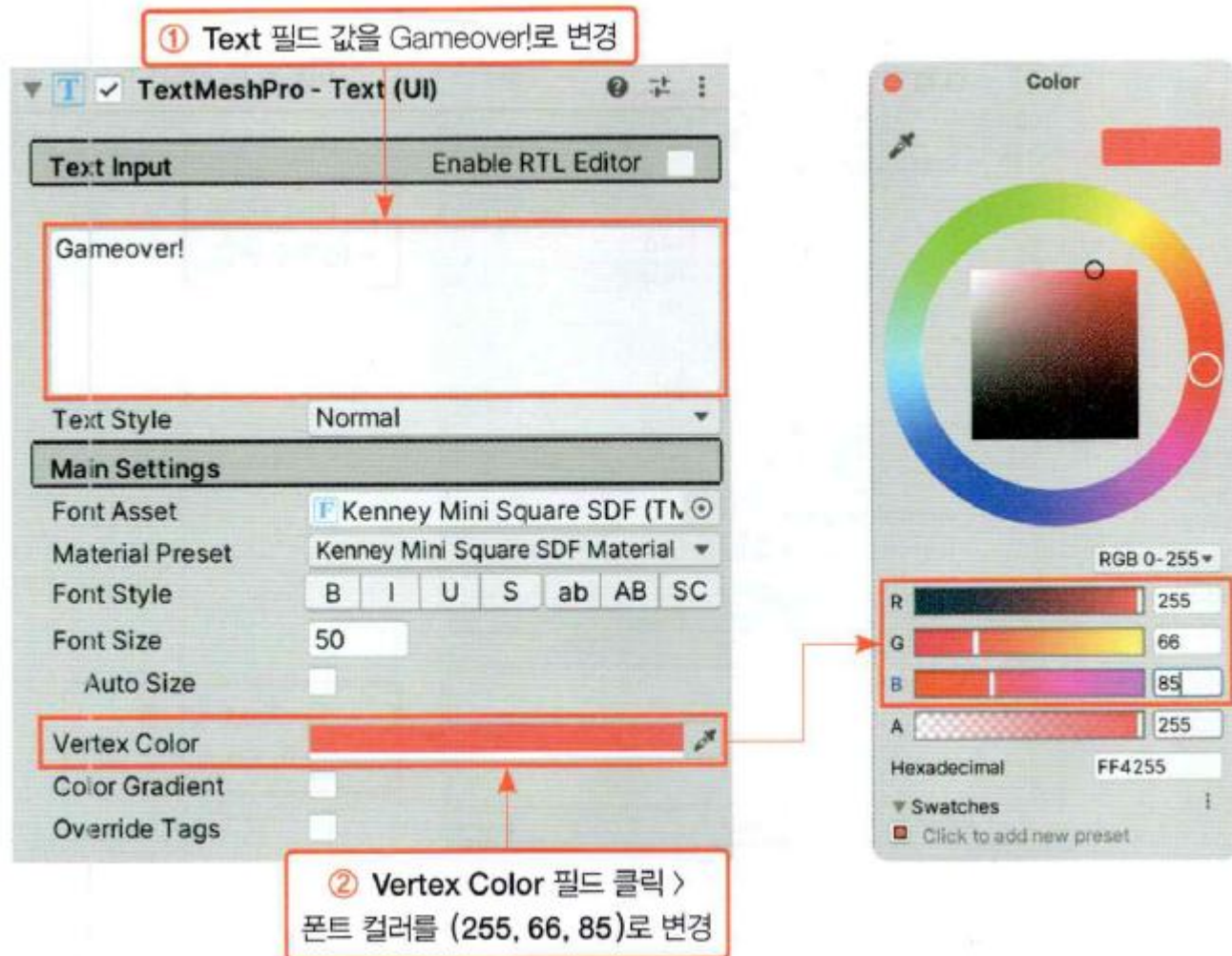
- ① 하이어라키 창에서 Score Text 선택 > [Ctrl+D]로 복제
- ② Score Text (1)의 이름을 Gameover Text로 변경
- ③ Anchor Preset 클릭 > [Alt+Shift]를 누른 채 top-center 클릭



12.3 게임 UI 만들기(11)

[과정 02] 게임오버 텍스트의 Text 컴포넌트 설정하기

- ① Text-TextMeshPro(UI) 컴포넌트의 Text 필드 값을 Gameover!로 변경
- ② Vertex Color 필드 클릭 > 폰트 컬러를 (255, 66, 85)로 변경



12.3 게임 UI 만들기(12)

- Restart Text 게임 오브젝트를 만들기
 - Gameover Text 아래에 게임 재시작 방법을 안내할 UI 텍스트

[과정 03] 게임 재시작 안내 텍스트 만들기

- ① 하이어라키 창에서 Gameover Text 선택 > [Ctrl+D]로 복제
- ② Gameover Text (1)의 이름을 Restart Text로 변경
- ③ Restart Text의 Rect Transform 컴포넌트의 Pos Y 값을 -80으로 변경
- ④ Text-TextMeshPro(UI) 컴포넌트의 Text 필드를 Jump To Restart로 변경
- ⑤ Font Size를 33으로 변경
- ⑥ Restart Text를 Gameover Text의 자식으로 만들기

12.3 게임 UI 만들기(13)

① Gameover Text 선택 > [Ctrl+D]로 복제

② 이름을 Restart Text로 변경

③ Pos Y값을 -80으로 변경

④ Text 필드를 Jump To Restart로 변경

⑤ Font Size를 33으로 변경

⑥ Restart Text를 Gameover Text의 자식으로 만들기 (Restart Text를 Gameover Text에 드래그&드롭)

12.3 게임 UI 만들기(14)

- Restart Text의 텍스트 컴포넌트는 'Jump To Restart!점프 버튼을 눌러 재시작하세요!'라는 메시지를 띄움



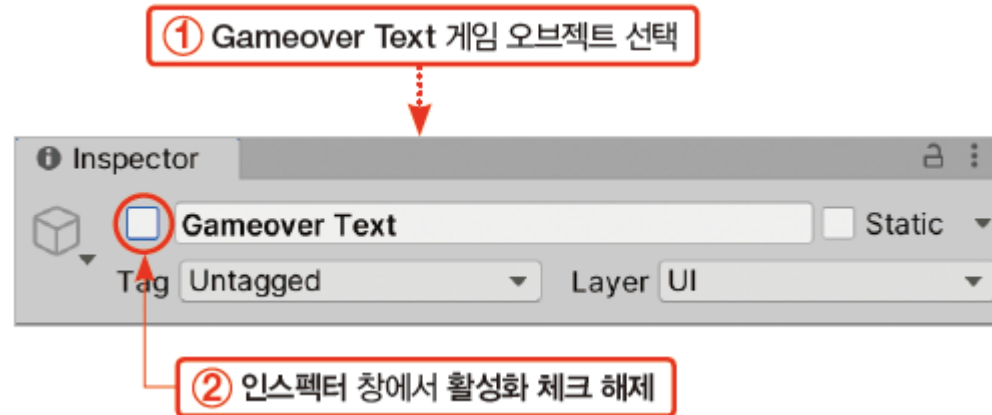
▶ 완성된 UI 모습

12.3 게임 UI 만들기(15)

- Gameover Text 게임 오브젝트는 평소에는 비활성화해 두었다가 게임오버가 된 순간 활성화
- Gameover Text 게임 오브젝트를 미리 비활성화

[과정 04] 게임오버 텍스트 비활성화하기

- ① 하이어라키 창에서 Gameover Text 게임 오브젝트 선택
- ② 인스펙터 창에서 활성화 체크 해제



12.4 게임 매니저 만들기(1)

- 게임 매니저의 역할
 - 점수 저장
 - 게임오버 상태 표현
 - 플레이어의 사망을 감지해 게임오버 처리 실행
 - 점수에 따라 점수 UI 텍스트 갱신
 - 게임오버되었을 때 게임오버 UI 활성화

12.4 게임 매니저 만들기(2)

12.4.1 싱글턴 패턴의 필요성

- 게임 매니저처럼 관리자 역할 오브젝트(예를 들면 파일 매니저, 몬스터 매니저, 점수 매니저 등)의 조건
 - 일반적으로 프로그램에 단 하나만 존재 (단일 오브젝트)
 - 언제 어디서든 즉시 접근 가능(손쉬운 접근)

단일 오브젝트

- 점수와 UI, 게임 상태를 관리하는 게임 매니저 또한 게임에 단 하나만 존재

손쉬운 접근

- 프로그램의 특정 영역이 아니라 어느 곳에서도 사용할 수 있는 편의 기능

싱글턴을 사용하는 이유

- '어떤 오브젝트가 프로그램에 단 하나만 존재해야 하며, 어느 곳에서도 쉽게 접근 가능해야 할 때' 사용

12.4 게임 매니저 만들기(3)

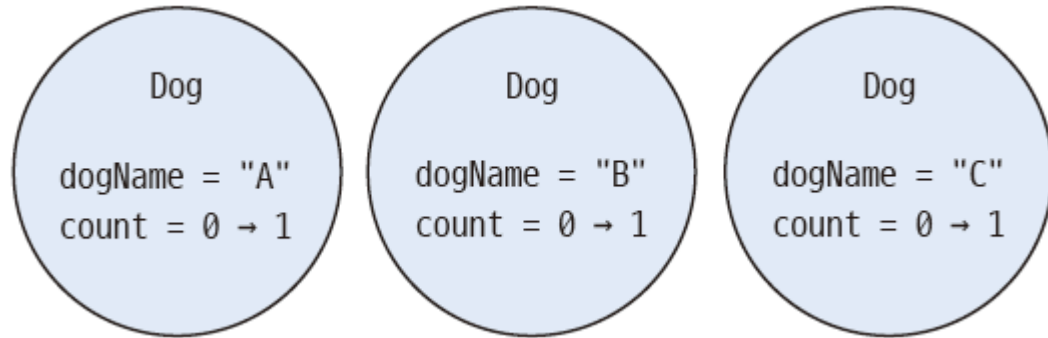
12.4.2 정적

- 싱글턴 패턴을 구현할 때는 정적 변수의 특징을 활용
- 어떤 Dog 클래스가 있다고 가정
 - Dog 오브젝트는 자신이 생성되고 활성화될 때 Awake() 메서드에 의해 count 값을 1 증가

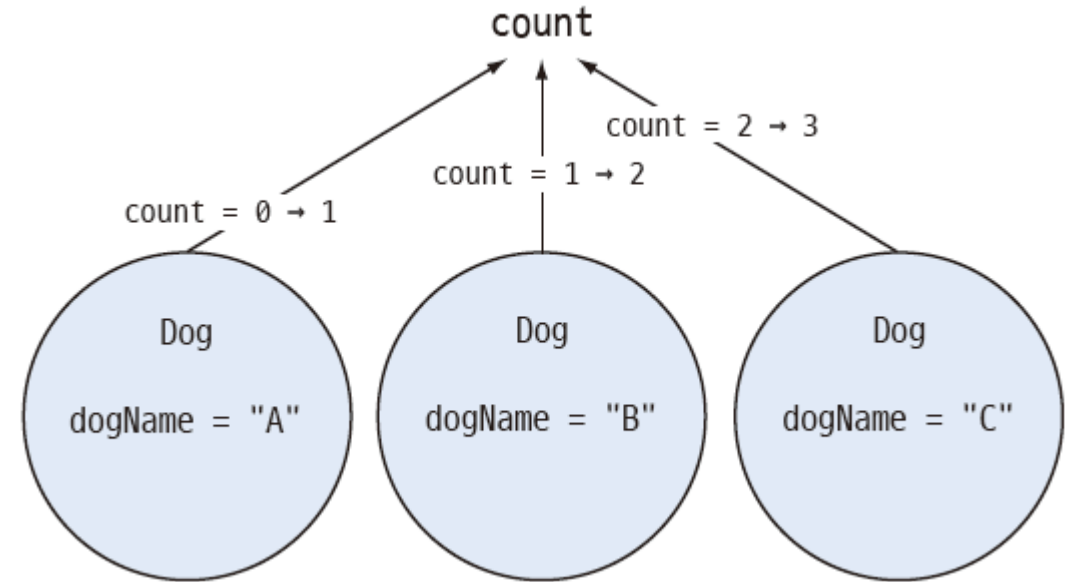
```
public class Dog : MonoBehaviour {  
    public string dogName; // 개의 이름  
    public int count = 0; // 프로그램에 존재하는 개의 총 수  
  
    void Awake() {  
        count++;  
    }  
}
```

12.4 게임 매니저 만들기(4)

- Dog 클래스로부터 Dog 타입의 오브젝트 A, B, C를 순서대로 생성했다고 가정할 때 count 값의 변화
 - 의도대로라면 count의 값은 3이 되어야 하지만 현재 각각의 Dog 오브젝트의 count 값은 1
 - count가 Dog 오브젝트의 총 수를 나타내기 위해서는 count가 Dog 오브젝트마다 하나씩 존재하면 안 됨
 - 아래 오른쪽 그림과 같이 count는 단 하나만 존재하고 모든 Dog 오브젝트가 하나의 count를 공유



▶ 각각의 Dog 오브젝트에서 count가 증가



▶ 하나의 count를 3개의 Dog가 공유

12.4 게임 매니저 만들기(5)

- 어떤 변수를 static으로 선언하면 여러 오브젝트가 해당 변수 하나를 공유
- 정적 선언된 변수는 해당 타입의 오브젝트를 몇 개를 생성하든 그 수에 상관없이 메모리에 하나만 존재

```
public class Dog : MonoBehaviour {  
    public string dogName; // 개의 이름  
    public static int count = 0; // 프로그램에 존재하는 개의 총 수  
  
    void Awake() {  
        count++;  
    }  
}
```

- Dog 스크립트 외부에서 Dog의 count에 새로운 값 100을 할당하려면 다음과 같은 코드를 작성

```
Dog.count = 100;
```

- static 변수의 특징
 - 메모리에 단 하나만 존재하고 모든 오브젝트가 공유
 - 클래스 이름과 점(.) 연산자를 이용해 접근

12.4 게임 매니저 만들기(6)

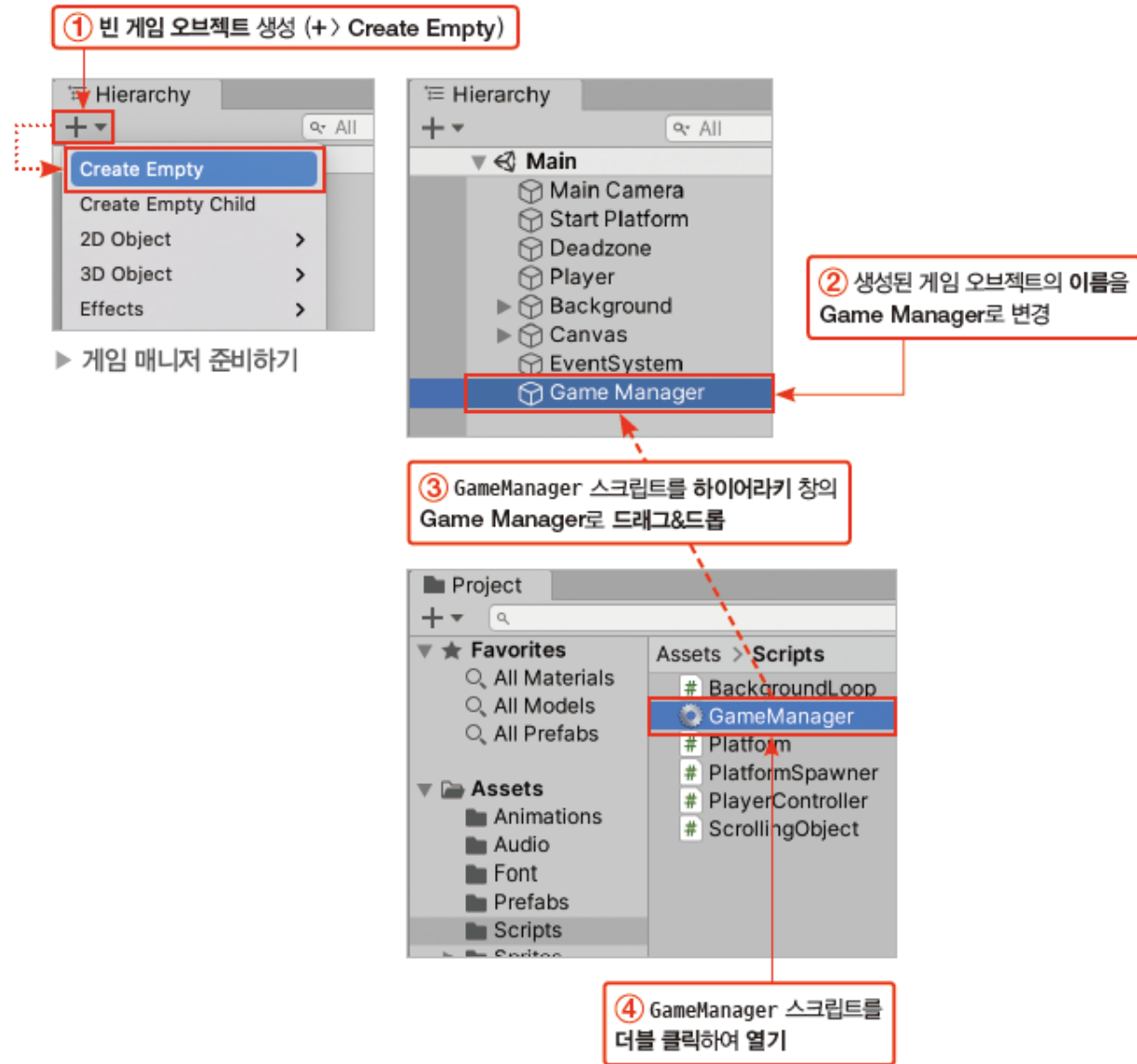
12.4.3 GameManager 스크립트 준비

- 게임 매니저가 될 게임 오브젝트를 만들고 GameManager 스크립트 열기

[과정 01] 게임 매니저 준비

- ① 빈 게임 오브젝트 생성(+ > Create Empty)
- ② 생성된 게임 오브젝트의 이름을 Game Manager로 변경
- ③ 프로젝트 창에서 Scripts 폴더의 GameManager 스크립트를 하이어라키 창의 Game Manager로 드래그&드롭
- ④ GameManager 스크립트를 더블 클릭하여 열기

12.4 게임 매니저 만들기(7)



12.4 게임 매니저 만들기(8)

12.4.4 GameManager 싱글턴 구현

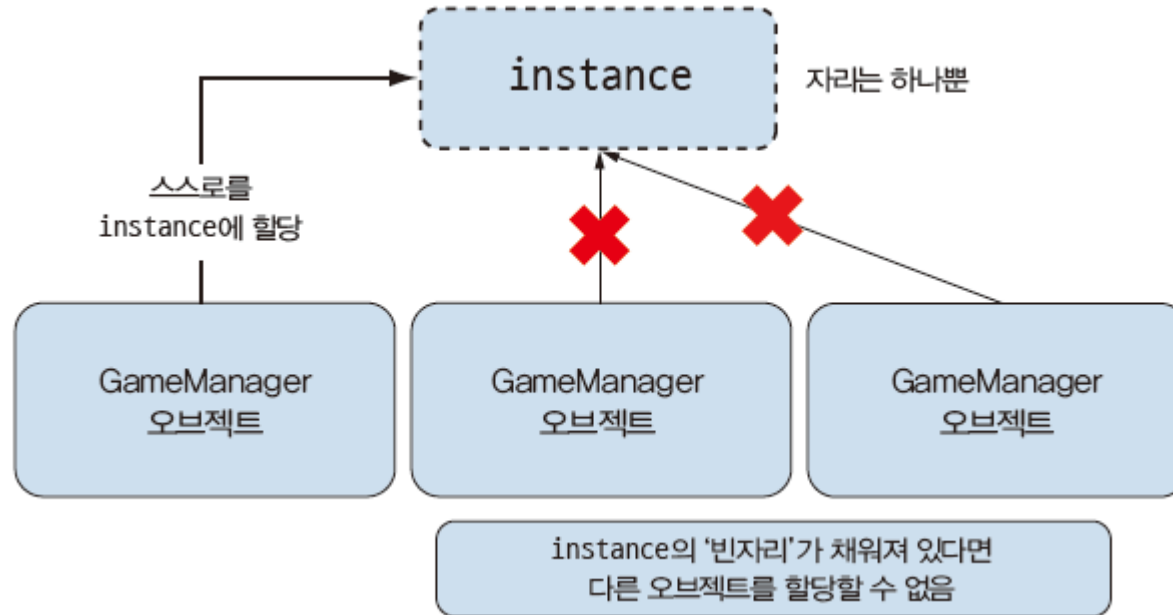
- GameManager에서 싱글턴을 구현한 부분을 먼저 살펴보고 다른 부분을 완성

```
public static GameManager instance; // 싱글턴을 할당할 전역 변수

void Awake() {
    if (instance == null)
    {
        instance = this;
    }
    else
    {
        Debug.LogWarning("썬에 두 개 이상의 게임 매니저가 존재합니다!");
        Destroy(gameObject);
    }
}
```

12.4 게임 매니저 만들기(9)

- 싱글톤이 될 GameManager 오브젝트는 스스로를 instance에 할당



- ▶ 단 하나의 GameManager 오브젝트만이 자신을 instance 변수에 할당할 수 있다

12.4 게임 매니저 만들기(10)

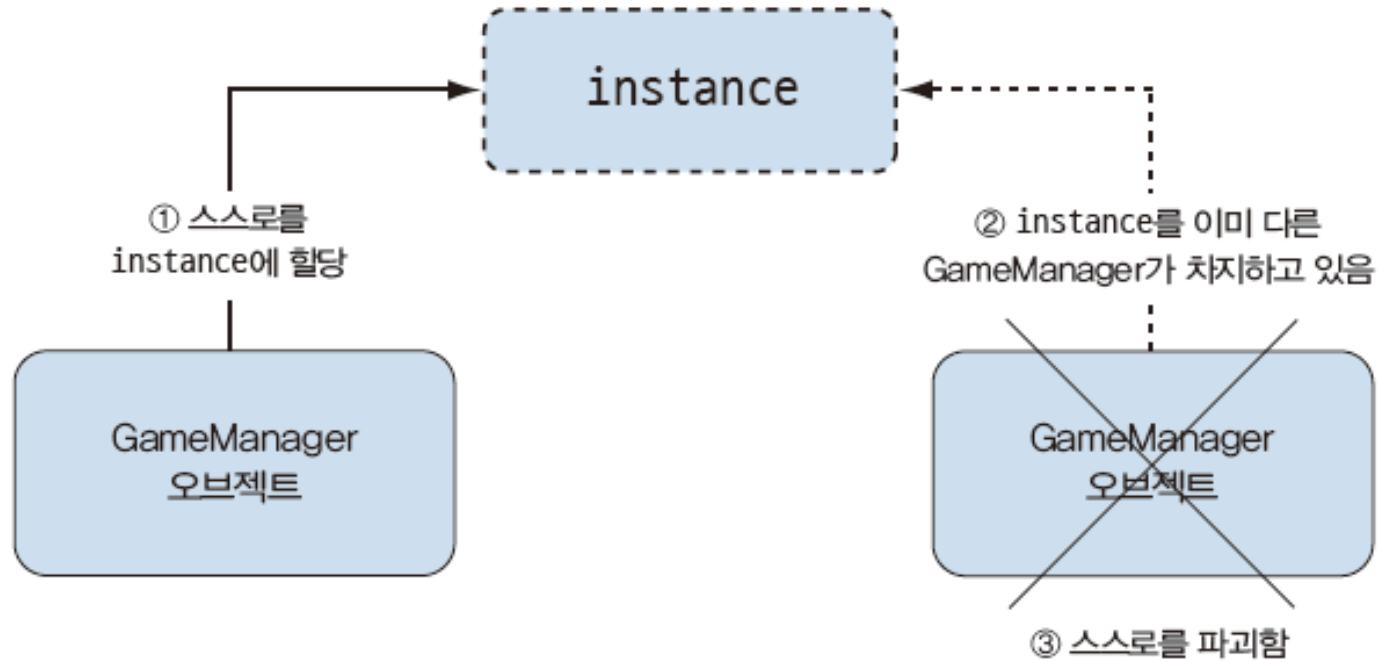
- Awake() 메서드의 if 문 블록에서는 instance가 비어 있다면 그곳에 자기 자신을 할당

```
if (instance == null)
{
    instance = this;
}
```

- else 문 블록은 instance가 null이 아닌 경우에 실행
 - 즉, 자신이 아닌 다른 GameManager 오브젝트가 instance에 이미 할당된 경우
 - 경고 로그를 출력하고 싱글톤이 될 수 없는 자신의 게임 오브젝트를 Destroy() 메서드로 파괴

```
if (instance == null)
{
    instance = this;
}
else
{
    Debug.LogWarning("썬에 두 개 이상의 게임 매니저가 존재합니다!");
    Destroy(gameObject);
}
```

12.4 게임 매니저 만들기(11)



▶ 스스로 파괴되는 두 번째 GameManager 오브젝트

12.4 게임 매니저 만들기(12)

12.4.5 GameManager의 필드

- instance 이외의 싱글톤 변수
 - isGameOver : 게임오버 상태
 - scoreText : 점수를 출력할 UI 텍스트 컴포넌트에 대한 참조
 - gameOverUI : 게임오버 시 활성화할 UI 게임 오브젝트에 대한 참조
 - score : 현재 게임 점수

```
using UnityEngine.SceneManagement;
```

```
using TMPro;
```

```
// 게임 오버 상태를 표현하고, 게임 점수와 UI를 관리하는 게임 매니저  
// 씬에는 단 하나의 게임 매니저만 존재할 수 있다.
```

```
public class GameManager : MonoBehaviour {  
    public static GameManager instance; // 싱글톤을 할당할 전역 변수  
  
    public bool isGameOver = false; // 게임 오버 상태  
    public TextMeshProUGUI scoreText; // 점수를 출력할 UI 텍스트  
    public GameObject gameOverUI; // 게임 오버시 활성화 할 UI 게임 오브젝트  
  
    private int score = 0; // 게임 점수
```


12.4 게임 매니저 만들기(13)

12.4.6 GameManager의 Update() 메서드

- GameManager 스크립트의 Update() 메서드에서 게임오버 상태일 때 게임을 재시작하는 기능 만들기
- 씬 관리자 SceneManager가 필요하므로 스크립트 상단에 다음과 같은 선언을 미리 추가

```
using UnityEngine.SceneManagement;
```

[과정 01] Update () 메서드 완성하기

- ① GameManager 스크립트의 Update() 메서드를 다음과 같이 완성

```
void Update() {  
    if (isGameOver && Input.GetMouseButtonDown(0))  
    {  
        // 게임오버 상태에서 마우스 왼쪽 버튼을 클릭하면 현재 씬 재시작  
        SceneManager.LoadScene(SceneManager.GetActiveScene().name);  
    }  
}
```

- 게임오버 상태에서 마우스 왼쪽 버튼을 누르면 if 문 블록 코드가 실행되고 현재 활성화된 씬을 다시 로드하여 게임을 재시작

```
SceneManager.LoadScene(SceneManager.GetActiveScene().name);
```

12.4 게임 매니저 만들기(14)

12.4.7 GameManager의 AddScore() 메서드

- 점수를 추가하는 AddScore() 메서드 구현
 - 입력으로 추가할 점수를 받아 입력된 값만큼 점수를 증가시키고, 점수 UI 텍스트에 출력되는 점수를 갱신

[과정 01] AddScore () 메서드 완성하기

① GameManager 스크립트의 AddScore() 메서드를 다음과 같이 완성

```
public void AddScore(int newScore) {  
    // 게임오버가 아니라면  
    if (!isGameOver)  
    {  
        // 점수를 증가  
        score += newScore;  
        scoreText.text = "Score : " + score;  
    }  
}
```

12.4 게임 매니저 만들기(15)

12.4.8 GameManager의 OnPlayerDead() 메서드

- 플레이어가 사망했을 때 실행할 처리를 구현하는 OnPlayerDead() 메서드를 완성
- 플레이어가 사망하면 게임오버 상태가 되며 게임오버 UI가 활성화

[과정 01] OnPlayerDead () 메서드 완성하기

① GameManager 스크립트의 OnPlayerDead() 메서드를 다음과 같이 완성

```
public void OnPlayerDead() {  
    isGameOver = true;  
    gameOverUI.SetActive(true);  
}
```

12.4 게임 매니저 만들기(16)

12.4.9 GameManager 스크립트 전체 코드

```
using UnityEngine;
using UnityEngine.SceneManagement;
using TMPro;

// 게임 오버 상태를 표현하고, 게임 점수와 UI를 관리하는 게임 매니저
// 씬에는 단 하나의 게임 매니저만 존재할 수 있다.
public class GameManager : MonoBehaviour {
    public static GameManager instance; // 싱글톤을 할당할 전역 변수

    public bool isGameOver = false; // 게임 오버 상태
    public TextMeshProUGUI scoreText; // 점수를 출력할 UI 텍스트
    public GameObject gameOverUI; // 게임 오버시 활성화 할 UI 게임 오브젝트

    private int score = 0; // 게임 점수
```

▶ 다음 쪽에 이어짐

12.4 게임 매니저 만들기(17)

◀ 앞쪽에 이어

```
// 게임 시작과 동시에 싱글톤을 구성
void Awake() {
    // 싱글톤 변수 instance가 비어있는가?
    if (instance == null)
    {
        // instance가 비어있다면(null) 그곳에 자기 자신을 할당
        instance = this;
    }
    else
    {
        // instance에 이미 다른 GameManager 오브젝트가 할당되어 있는 경우

        // 썬에 두개 이상의 GameManager 오브젝트가 존재한다는 의미.
        // 싱글톤 오브젝트는 하나만 존재해야 하므로 자신의 게임 오브젝트를 파괴
        Debug.LogWarning("썬에 두개 이상의 게임 매니저가 존재합니다!");
        Destroy(gameObject);
    }
}
```

▶ 다음 쪽에 이어짐

12.4 게임 매니저 만들기(18)

◀ 앞쪽에 이어

```
void Update() {  
    // 게임 오버 상태에서 게임을 재시작할 수 있게 하는 처리  
    if (isGameOver && Input.GetMouseButtonDown(0))  
    {  
        SceneManager.LoadScene(SceneManager.GetActiveScene().name);  
    }  
}  
  
// 점수를 증가시키는 메서드  
public void AddScore(int newScore) {  
    if (!isGameOver)  
    {  
        score += newScore;  
        scoreText.text = "Score : " + score;  
    }  
}
```

▶ 다음 쪽에 이어짐

12.4 게임 매니저 만들기(19)

◀ 앞쪽에 이어

```
// 플레이어 캐릭터가 사망시 게임 오버를 실행하는 메서드
public void OnPlayerDead() {
    isGameOver = true;
    gameOverUI.SetActive(true);
}
}
```

12.4 게임 매니저 만들기(20)

12.4.10 PlayerController 스크립트 수정

- OnPlayerDead() 메서드는 PlayerController 스크립트의 Die() 메서드에서 실행
- PlayerController 스크립트의 Die() 메서드를 수정

[과정 01] PlayerController 스크립트 수정하기

① PlayerController 스크립트를 열고 Die() 메서드를 다음과 같이 수정

```
private void Die() {  
    // 애니메이터의 Die 트리거 파라미터를 셋  
    animator.SetTrigger("Die");  
  
    // 오디오 소스에 할당된 오디오 클립을 deathClip으로 변경  
    playerAudio.clip = deathClip;  
    // 사망 효과음 재생  
    playerAudio.Play();  
  
    // 속도를 제로(0, 0)로 변경  
    playerRigidbody.velocity = Vector2.zero;  
    // 사망 상태를 true로 변경  
    isDead = true;  
  
    // 게임 매니저의 게임오버 처리 실행  
    GameManager.instance.OnPlayerDead();  
}
```


12.4 게임 매니저 만들기(21)

12.4.11 ScrollingObject 스크립트 수정

- 게임오버인 상태에서는 움직이던 발판과 배경이 멈춰야 함.
 - GameManager의 변수 isGameOver가 게임오버 상태를 나타내고 있으므로 ScrollingObject 스크립트에서 GameManager의 isGameOver를 검사하여 움직임 여부를 결정

[과정 01] ScrollingObject 스크립트 수정하기

① ScrollingObject 스크립트를 열고 Update() 메서드를 다음과 같이 수정

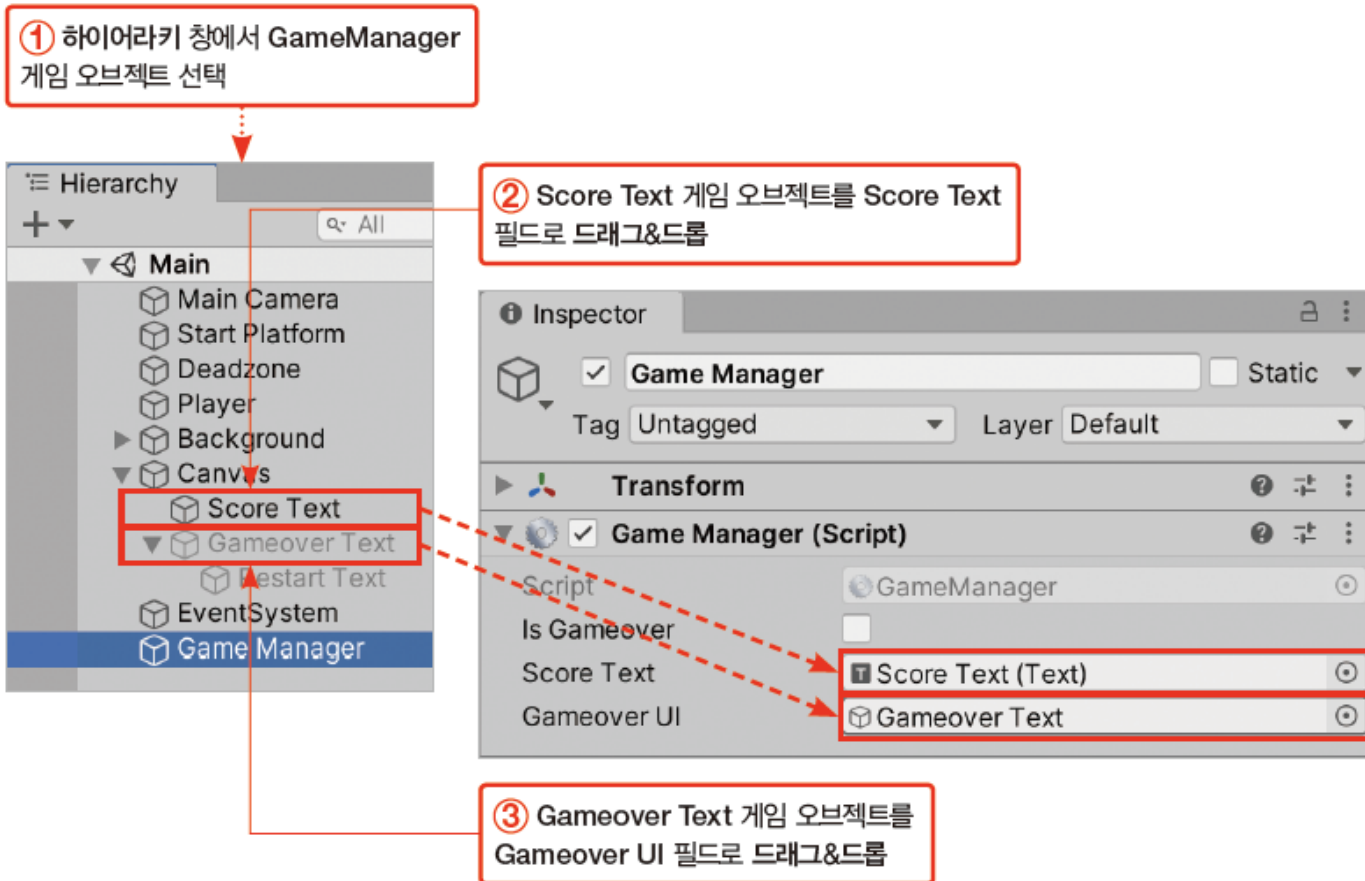
```
private void Update() {  
    // 게임오버가 아니라면  
    if (!GameManager.instance.isGameOver)  
    {  
        // 초당 speed의 속도로 왼쪽으로 평행이동  
        transform.Translate(Vector3.left * speed * Time.deltaTime);  
    }  
}
```

12.4 게임 매니저 만들기(22)

12.4.12 GameManager 컴포넌트 설정하기

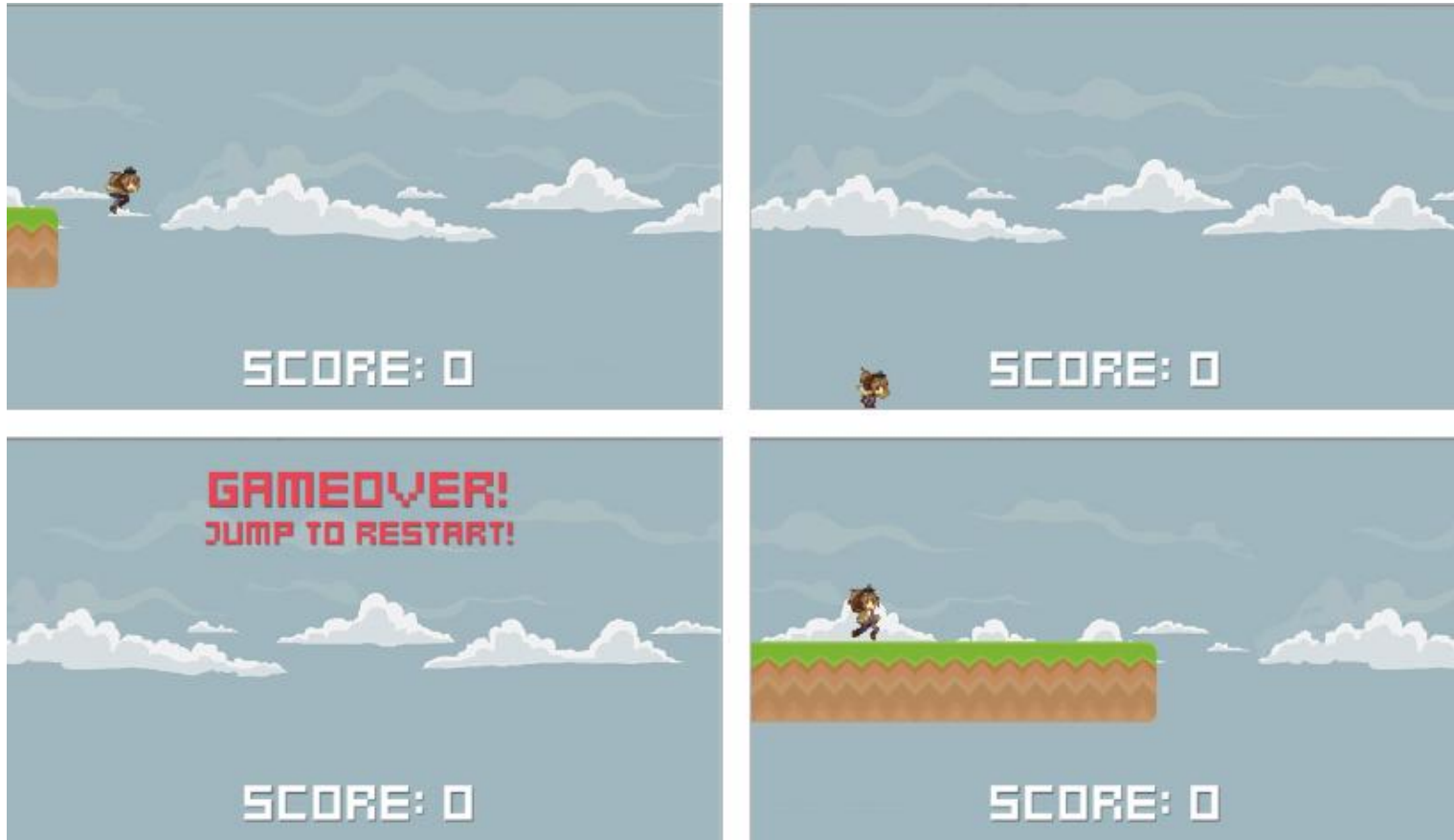
[과정 01] GameManager 컴포넌트 설정하기

- ① 하이어라키 창에서 GameManager 게임 오브젝트 선택
- ② 하이어라키 창의 Score Text 게임 오브젝트를 GameManager 컴포넌트의 Score Text 필드로 드래그&드롭
- ③ 하이어라키 창의 Gameover Text 게임 오브젝트를 GameManager 컴포넌트의 Gameover UI 필드로 드래그&드롭



12.4 게임 매니저 만들기(23)

- 플레이 버튼을 눌러 실행 테스트
 - 플레이어 캐릭터가 낙사하면 게임오버 UI가 활성화되며 더 이상 배경이나 발판이 움직이지 않는 것을 확인
 - 게임오버 상태에서 마우스 왼쪽 버튼을 누르면 게임이 다시 시작



12.5 마치며

이 장에서 배운 내용 요약

- 2D 게임 오브젝트 그리기 순서는 정렬 레이어로 조정
- 캔버스 스케일러 컴포넌트의 스케일 모드는 다양한 화면 크기에 대해 캔버스의 크기가 어떻게 변경될지 결정
- '고정 픽셀 크기' 모드에서는 화면 크기가 달라져도 UI 요소의 크기는 달라지지 않음
- '고정 픽셀 크기' 모드에서는 화면의 크기가 커졌을 때 UI가 상대적으로 작게 보이는 문제가 있음
- '화면 크기에 따른 스케일' 모드에서는 기준 해상도를 정하고, 게임 화면 크기에 맞춰 캔버스를 확대/축소하는 방식
- '화면 크기에 따른 스케일' 모드에서는 다양한 크기와 비율의 화면에서도 기준 해상도에서의 UI 모습을 최대한 유지
- static으로 선언된 정적 변수는 하나의 변수를 여러 오브젝트가 공유하는 방식으로 동작
- 정적 변수는 클래스 이름과 점 연산자를 이용해서 접근
- 싱글턴 패턴은 '단 하나만 존재해야 하며, 손쉽게 접근 가능한' 오브젝트를 구현하는 디자인 패턴